RapidBoxForest: Link-Envelope-Guided Box Forests for Low-Cost Scene Builds in Multi-Query Manipulation

TIAN, Yu and Ren, Hongliang

***Abstract*—Repeated-query manipulation benefits from reusable free-space structure, but high-quality convex regions can be costly to rebuild after local scene edits. RapidBoxForest (RBF) uses endpoint-to-link swept-capsule bounds as a configuration-space box-validation oracle, validating volumetric joint boxes rather than individual path segments. Accepted boxes are organized as a scene-level box forest and typed corridor graph for repeated start-goal queries, while a Lifelong Envelope Cache Tree (LECT) stores kinematics-keyed evidence separately from obstacle-dependent scene labels. The certificate is conditional: paths inside conservatively validated box unions inherit it, whereas tighter filters, repair, or post-processing require independent query validation. Saved-catalog, audit-separated experiments demonstrate low warm-replay cost on a KUKA LBR iiwa shelf-workcell benchmark (Shelf+IIWA), live-materialization cache context, and isolated local-maintenance cost under that validation policy; external cache precompute/load and end-to-end replanning are outside the reported timing claims.**

***Index Terms*—Motion planning, multi-query planning, link interval envelopes, box-region planning, configuration-space planning.**

I. Introduction

High-degree-of-freedom (high-DOF) manipulators often face many related queries in nearly fixed workcells: the robot kinematics persist, while start-goal pairs, local obstacles, or small scene edits change. Reuse is valuable only if the reusable object is inexpensive enough to build, locally maintain, and query.

Convex-region methods such as IRIS and Graphs of Convex Sets provide powerful region-level planning abstractions once certified regions are available [1], [2], [3], but their construction cost can dominate in high-dimensional or frequently edited scenes. Sampling-based methods such as PRM and RRT variants avoid heavy preprocessing and solve single queries effectively [4], [5], [6], [7], yet their reusable structures are usually zero-volume roadmaps or trees.

RBF targets the middle ground: it grows axis-aligned configuration-space (C-space) boxes, validates them with a link-envelope scene oracle, connects accepted boxes into a graph, and answers later queries by searching the resulting corridor. The design trades region expressivity and path optimality for low scene-build cost, simple adjacency, and explicit multi-query reuse. The resulting planning object is intended for regimes in which repeated-query latency and rebuild cost matter more than obtaining a tighter regional decomposition.

a) Problem statement: Let $\mathcal{Q} \subset \mathbb{R}^n$ be the joint-limit box of a fixed manipulator and let $\mathcal{O} \subset \mathbb{R}^3$ be the current workspace obstacle set. Given a batch or stream of start-goal queries $(q_s, q_g) \in \mathcal{Q}^2$, the objective is to build a scene-specific planning object that supports repeated query retrieval while amortizing geometric work that depends only on robot kinematics and joint intervals. The object sought by RBF is a finite family of validated C-space boxes and an adjacency graph over those boxes. The design target is low scene rebuild and isolated local-maintenance cost relative to heavier region construction, while still exposing volumetric free-space structure rather than only path-local collision checks.

The key geometric ingredient is not a new capsule swept-volume identity, but a new use of the classical endpoint-bounding pipeline. For rounded link models, RBF separates the swept capsule into the sweep of a zero-radius center segment and a final radius lift. Endpoint interval envelopes bound the two endpoint trajectories over a joint box; the convex hull of these endpoint envelopes, enlarged by the link radius, encloses the swept rounded link. RBF uses this bound as a C-space box free-space validation oracle. The same endpoint evidence can be exported as axis-aligned bounding boxes (AABBs), SupportHull records, or staged AABB–SupportHull filters. Conservative endpoint sources yield a conditional conservative box certificate, whereas tighter practical sources act as planning filters under the common query-validation policy.

The planning layer builds directly on this primitive. A sample-guided grower selects query-relevant frontiers, materializes local candidate boxes around projected seeds, and extends a connected forest of validated regions. This is a workload-biased adaptive cell decomposition in manipulator C-space. RBF has two reuse levels. The scene-level RBF cache is the constructed box forest and corridor graph for a fixed obstacle set; it supports repeated start-goal retrieval in that environment. The lifelong envelope cache, LECT, memoizes scene-independent endpoint and link-envelope evidence keyed by robot kinematics and joint intervals; when cache descriptors match, it reduces compatible future build or local-refill materialization work,

The authors are with the Department of Electronic Engineering, The Chinese University of Hong Kong (CUHK), Hong Kong SAR, China. E-mail: ty1997@link.cuhk.edu.hk.

TABLE I
Qualitative taxonomy of reusable manipulation-planning paradigms. The comparison emphasizes the persistent object, validation granularity, and scene-edit semantics rather than ranking planners by runtime.

Family	Persistent object	Vol.	Validation / certificate unit	Scene-edit reuse	Main trade-off
PRM / LazyPRM	Roadmap vertices and edges	No	Local edges or path segments	Topology can be reused; affected edges require rechecking	Effective multi-query graph, but reusable evidence is zero-volume.
RRTConnect / RRT	Query tree	No	Incremental path segments	Little persistent reuse across queries	Strong one-shot discovery; persistent structure is query-local.
BIT* / FMT*	Batch sample graph or tree	No	Candidate graph edges	Mainly query-centric; limited scene-level reuse	Anytime quality improvement, but still edge-centric.
Adaptive cell decomposition	Configuration-space cells	Yes	Cells under exact, sampled, or interval tests	Explicit cells can be locally relabeled when maintained	Volumetric representation; high-dimensional manipulation needs a cheap cell-validation oracle.
IRIS / GCS	Certified convex regions and graph	Yes	Convex free-space regions	Regions are mostly scene-coupled; edits may require reinflation or recertification	Optimization-ready regions with higher construction and edit cost.
RBF	Box forest + typed graph + LECT evidence	Yes	Joint boxes via link envelopes; final audit when required	Scene boxes are maintained locally; LECT replays kinematic evidence	Low-build volumetric reuse with lower regional expressivity and conditional certificates.

Abbrev.: PRM=probabilistic roadmap; RRT=rapidly exploring random tree; BIT*/FMT*=batch-informed/fast-marching trees; IRIS=iterative regional inflation; GCS=Graphs of Convex Sets; LECT=Lifelong Envelope Cache Tree; Vol.=volumetric.

while free-space labels remain scene-dependent.

The evaluation is organized around this design point, with three explicit boundaries. Main successes are final-query-validation results; the theorem applies only to box-contained conservative corridors. Low Shelf build times are warm-replay scene builds: snapshot size and prewarm time are reported, but snapshot load and resident-set-size (RSS) memory accounting and full precompute amortization are not. Update rows are isolated scene-cache maintenance, not end-to-end replanning. Thus the claim is low-cost reusable region construction under a stated validation policy, not universal single-query dominance, conservative-only completeness, or full dynamic planning.

RBF is therefore a planning-systems contribution enabled by a geometric primitive. The endpoint-to-link envelope supplies the C-space box-validation oracle; the scene-level box-forest cache, lifelong LECT evidence cache, and local maintenance rules show how that oracle can be assembled into a reusable multi-query planning pipeline.

The paper contributes RBF**, a link-envelope-guided box-region planner that lifts classical swept-capsule endpoint bounds from collision detection to reusable C-space free-space construction.** The first item is the geometric/formal contribution, the next three are system contributions, and the last item is the evaluation contribution:

1) *Geometric primitive reuse: joint-box link-envelope validation.* RBF repurposes continuous-collision-detection (CCD) endpoint bounds as a joint-box free-space oracle, with conservative sources for certificates and tighter staged sources for query-validated filtering (section III).
2) *Planning system: box forest and typed corridor graph.* Seed descent, leaf-sweep coverage, restricted refinement, frontier expansion, and consolidation produce a scene-level typed corridor graph of validated boxes (section V).
3) *Reuse architecture: scene cache versus LECT evidence cache.* Scene RBF caches store the current environment's box forest, while LECT stores lifelong kinematics-keyed endpoint and link-envelope evidence reusable across builds (sections IV and V).
4) *Local maintenance: deletion promotion and insertion refill.* Deletions revalidate formerly blocked domains for promotion, and insertions invalidate conflicting boxes before bounded local refill (section V-G).
5) *Evaluation protocol: saved catalogs and audit-separated timing.* The experiments separate build, online solve, final simplification, and strict audit time, keep saved scene/query catalogs fixed across planners, and report cache reuse only when the corresponding LECT evidence is explicitly enabled (section VI).

The rest of the paper develops the envelope oracle (section III), LECT reuse layer (section IV), box forest and update rules (section V), and evaluation (section VI).

II. Related Work

Convex-region planners construct certified free-space covers and then solve graph search or convex optimization on the induced graph. IRIS and its successors compute large collision-free convex regions by alternating separating hyperplanes, inflation, visibility or clique-cover graph construction, and configuration-space refinement [1], [8], [9], [10]; Graphs of Convex Sets made this view especially influential for motion planning [2], [3]. RBF shares the region-level objective but deliberately uses axis-aligned C-space boxes: the cover is less expressive than rich polytopes, but validation, adjacency, and local maintenance have lower construction overhead. Table I summarizes this qualitative trade-off against roadmap, tree, sample-graph, adaptive-cell, and convex-region reuse.

a) Continuous-collision-detection endpoint bounds: Swept-volume approximation and interval forward kinematics provide the geometric antecedents. For a convex body, bounding generator trajectories and taking their convex hull outer-bounds the sweep [11], [12], [13]. For capsule links, the Minkowski identity $C = S \oplus B_2(r)$ reduces the sweep to the zero-radius segment sweep plus a radius lift; the Proximity Query Package (PQP), Gilbert–Johnson–Keerthi (GJK), and the Flexible Collision Library (FCL) exploit the same swept-sphere structure for

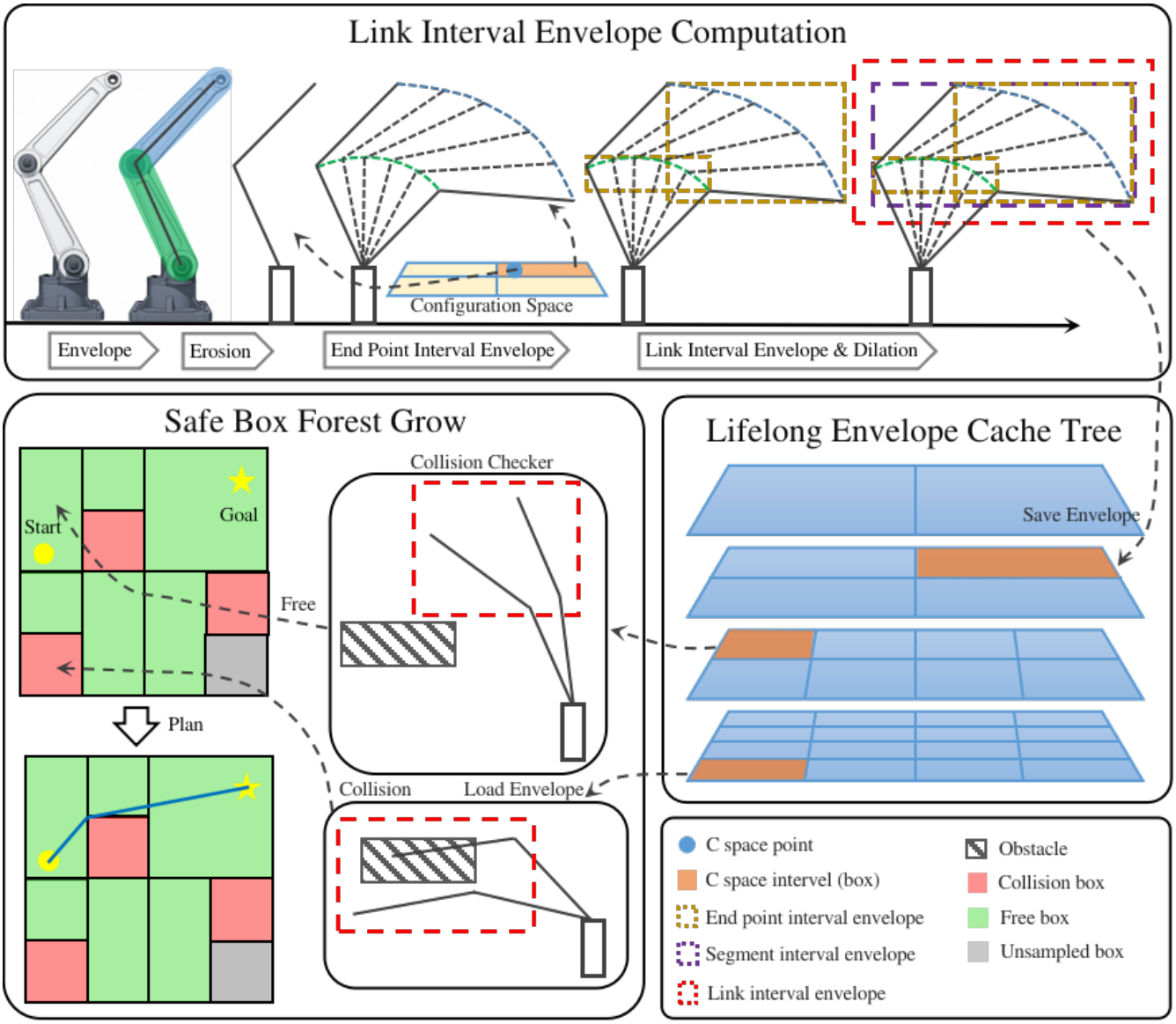


Fig. 1. RBF pipeline for link-envelope-guided multi-query planning. Top: endpoint interval envelopes induce link interval envelopes over a configuration-space (C-space) box. Middle: LECT reuses kinematics-keyed endpoint and link-envelope records across repeated builds. Bottom: scene filtering and sample-guided growth expand a connected scene-level box forest for repeated corridor queries.

proximity queries [14], [15], [16], [17]. Interval arithmetic and Denavit–Hartenberg (DH) chain interval propagation give inclusion-monotone endpoint boxes over parameter intervals [18], [19], [20], [21], [22]. Redon *et al.* apply exactly this endpoint-bounding pipeline to articulated-body CCD over a time step, padding the two bounded endpoint positions by the link radius [23], [24]; Taylor-model, path-evidence, and human-safety variants use related swept-sphere bounds [25], [26], [27]. The novelty boundary is therefore not the capsule identity itself. RBF changes the parameter domain from a trajectory time step to a joint-box interval, uses switchable endpoint sources and compact link records for staged box filtering, stores compatible evidence in a persistent kinematics-keyed LECT cache, and assembles accepted boxes into a typed corridor graph. Query validation remains the acceptance rule for repaired or post-processed paths. Operationally, continuous collision detection consumes endpoint bounds for one motion segment, whereas RBF stores interval evidence for later scenes and rebuilds scene labels and corridor connectivity. Only paths that remain inside conservative box unions inherit the corridor theorem; all repaired or segment-witness paths are query validated.

b) Reuse and adaptive decomposition: Multi-query planners amortize effort through reusable graphs. PRM and LazyPRM reuse roadmaps and cached edge checks [6], [28], [29], [30], while experience-based systems reuse prior paths or local snippets as query bias [31]. Classical adaptive cell decompositions split cells where evidence is insufficient and search the accepted-cell adjacency graph [32], [33]. RBF combines these ideas in high-dimensional manipulator C-space: scene-level boxes and corridor edges are reused inside one obstacle set, while kinematics-keyed envelope records can be replayed across compatible scene edits and future builds.

c) Sampling baselines and refinement: RRTConnect remains a strong one-shot manipulator baseline; RRT*, BIT*, FMT*, and benchmarking libraries provide anytime or sample-graph alternatives [34], [7], [35], [36], [37], [38], [39]. These methods solve single queries well but usually leave zero-volume, query-local evidence. Trajectory optimizers refine a supplied path [40], [41], [42], [43]; RBF is complementary, providing a reusable region graph before downstream repair, smoothing, and final path checking.

III. Link Interval Envelopes

This section defines the geometric primitive used by RBF: a reusable map from a joint-space region to compact link interval envelopes. The default region is an axis-aligned joint box, while query-critical bridge repairs may use a local oriented box in scaled joint coordinates. The goal is not exact swept volume computation, but a low-cost outer representation for box growth. Strict endpoint sources give conservative certificates; tighter sampled or support-based sources act as filters before query validation.

Let $\mathcal{Q}$ be the joint-limit box, $I \subseteq \mathcal{Q}$ a joint interval, and $\mathcal{O} \subset \mathbb{R}^3$ the obstacle set. Endpoint envelope $E_k(I)$ outer-bounds endpoint k, and link envelope $B_\ell(I)$ outer-bounds link ℓ over I, yielding the primitive $I \mapsto \{E_k(I)\} \mapsto B_\ell(I)$. Conservative endpoint evidence gives a conservative link envelope; empirical evidence is used only as a filter.

Assumption 1 (Collision model for certificates)**.** *The formal certificates in this paper concern collision between the configured active rounded links and the current workspace obstacle set $\mathcal{O}$. Joint limits are enforced by $I \subseteq \mathcal{Q}$. Self-collision, attached objects, grippers, or task-specific forbidden regions inherit a box-level certificate only if their corresponding link-link, link-object, or forbidden-region checks are part of the scene-validation policy and, when they change geometry or active-link sets, the cache key. The reported experiments use the active-link–workspace obstacle model only; they should not be read as full manipulation-stack certificates for carried objects, grippers, or self-collision.*

A. Geometric Foundation

Let $T(q)x = R(q)x + t(q)$ denote the rigid transform induced by a configuration $q \in I$. For a reference body $C_0 \subset \mathbb{R}^3$, its pose at q is $C(q) = T(q)C_0$. We write $\mathcal{S}_I(C_0)$ for the exact workspace set swept as q ranges over the joint box I and $\mathcal{C}_I(C_0)$ for its convex envelope. RBF usually works with the convex envelope because the downstream representations are AABBs or support-function hull records. The construction below deliberately reuses two classical facts—capsule Minkowski decomposition and convex-hull generator bounding—that already appear in swept-sphere collision libraries and articulated-body CCD work [23], [24]. The change is the domain and downstream object: the parameter set is a C-space joint box, and the output is a candidate box-region for a planning graph.

Radius isolation. For a capsule $C = S \oplus B_2(r)$, rigid motion commutes with the fixed Euclidean ball, giving exact two-stage sweep bounds:

$$\mathcal{S}_I(C) = \mathcal{S}_I(S) \oplus B_2(r), \qquad \mathcal{C}_I(C) = \mathcal{C}_I(S) \oplus B_2(r).$$

RBF therefore follows the same centerline-first, radius-lift construction used by swept-sphere collision methods. This identity is exact for capsules; for sharp polytopes it requires the shape to be a rounded offset body.

Convex-hull generator bound. For a convex body $P = \mathrm{conv}(v_1, \ldots, v_m)$, bounding each vertex trajectory by an outer set $\widehat{\Gamma}_i(I)$ suffices to bound the swept polytope:

$$\mathcal{S}_I(P) \subseteq \mathrm{conv}\Big(\bigcup_i \widehat{\Gamma}_i(I)\Big).$$

For a capsule link the seed is the center segment $[a, b]$, so only two endpoint trajectories need to be bounded.

Combining these two facts yields the practical recipe. For a capsule link $[a, b] \oplus B_2(r)$, RBF bounds only the two endpoint trajectories over the joint box, forms their convex hull, and applies the radius lift:

$$\widehat{\mathcal{C}}_I([a, b] \oplus B_2(r)) = \mathrm{conv}\big(\widehat{\Gamma}_a(I) \cup \widehat{\Gamma}_b(I)\big) \oplus B_2(r).$$

The numerical task reduces to computing reusable envelopes for two endpoint trajectories per link over I. Those records are then used by the planner to classify I as a candidate box-region, not merely to answer a path-segment CCD query.

Proposition 1 (Endpoint envelopes induce link interval envelopes)**.** *Consider a rounded link $L = [a, b] \oplus B_2(r)$ and a joint interval I. If endpoint envelopes $E_a(I)$ and $E_b(I)$ satisfy $\Gamma_a(I) \subseteq E_a(I)$ and $\Gamma_b(I) \subseteq E_b(I)$, then*

$$\mathcal{S}_I(L) \subseteq \mathrm{conv}\big(E_a(I) \cup E_b(I)\big) \oplus B_2(r).$$

Consequently, any outer representation of the right-hand side is a valid link interval envelope for the whole joint box I.

Proof. For each $q \in I$, the center segment of the moved link is the convex hull of the two moved endpoints, so $T(q)[a, b] \subseteq \mathrm{conv}(\Gamma_a(I) \cup \Gamma_b(I))$ over the entire interval. The endpoint inclusions place this set inside $\mathrm{conv}(E_a(I) \cup E_b(I))$. Adding the fixed radius ball commutes with the rigid-motion sweep of the capsule, giving the stated containment. □

Corollary 1 (Conservative joint-box validation)**.** *Let $I \subseteq \mathcal{Q}$ be a joint box and let $\{B_\ell(I)\}$ be conservative link interval envelopes for all active rounded links. If $B_\ell(I) \cap \mathcal{O} = \emptyset$ for every link ℓ, then every configuration $q \in I$ is collision-free with respect to the collision model in assumption 1.*

Proof. By proposition 1, each conservative link envelope contains the swept workspace volume of its link over all configurations in I. Disjointness from $\mathcal{O}$ for every active link therefore excludes workspace obstacle collision for the whole manipulator at every $q \in I$ under assumption 1. □

a) Numerical conservatism: The certificate above is a mathematical containment statement. A row or mode

is claimed to inherit it only when endpoint propagation, envelope padding, and obstacle-disjointness tests are configured as outward bounds under the stated collision model. In practice this requires interval or affine-arithmetic rounding/padding that covers floating-point residuals, conservative treatment of obstacle and link radii, and collision-test tolerances that cannot turn an intersecting envelope into a disjoint one. AABB radius lifts use coordinatewise B_∞ padding, which is looser than Euclidean capsule padding but remains an outer bound; SupportHull/GJK inherits a certificate only when obstacle inflation, safety margins, and termination tolerances are configured as outward tests. Empirical endpoint sources, uncovered SupportHull/GJK tolerances, or post-processing outside the conservative box union are therefore treated as filters whose returned paths must pass the independent query-validation path; they are not used to extend corollary 1 and theorem 1.

B. Endpoint Interval Envelopes

Figure 1 illustrates the endpoint-to-link envelope extraction pipeline, LECT replay, and the scene forest. Endpoint interval axis-aligned bounding boxes (iAABBs) are computed by interval forward kinematics (IFK) or tighter certified affine-arithmetic (AA)-based variants, combined into a conservative zero-radius centerline envelope, and padded by the link radius before collision querying. The remaining numerical task is to outer-bound each endpoint trajectory over I by a reusable set $E_k(I)$. Endpoint envelopes are the common interface between kinematic propagation and the downstream link-envelope representations.

For endpoint k, let $\Gamma_k(I)$ denote its exact workspace trajectory over interval I. Any conservative set that contains $\Gamma_k(I)$ is an endpoint interval envelope. The basic endpoint record stores this envelope as an axis-aligned bounding box, because boxes are efficient to store, easy to combine, and directly reusable by all downstream link representations.

a) Interval forward kinematics (IFK): The IFK endpoint source used by the planner is backed by affine-arithmetic forward kinematics. It propagates workspace bounds along the DH chain and provides the lowest-cost certified member of the endpoint-source family. Its limitation is the usual wrapping effect: on wide boxes or long chains, one evaluation accumulates residual over-approximation across the full joint box.

b) Hierarchical IFK (HIFK): HIFK further reduces wrapping by recursively bisecting the joint box and evaluating affine-arithmetic forward kinematics at the leaves before hulling the child endpoint boxes. HIFK is therefore a distinct endpoint source rather than a naming variant of unsplit IFK: it trades additional forward-kinematics evaluations for tighter endpoint boxes, while the exact split level is treated as an endpoint-source setting recorded in the artifacts.

c) Incremental forward-kinematics state reuse: Tree descent also carries a transient forward-kinematics state: interval-valued prefix transforms $[\boldsymbol{T}^{0:0}], \ldots, [\boldsymbol{T}^{0:n}]$ and the per-joint DH interval matrices used to build them. The full prefix chain is computed once at the root or after a restart; when only I_k changes, prefixes before k remain valid and only the suffix is recomputed. This short-lived state is not the persistent cache itself; it only keeps endpoint materialization efficient before LECT decides whether to retain the evidence.

Taken together, these sources define a clear certified spectrum: unsplit AA-based IFK is the lowest-cost source, while HIFK offers tighter outer bounds at higher cost. Endpoint envelopes are the right intermediate abstraction for the planner because they isolate the kinematic bounding problem from the scene-query and graph-building problems. The downstream link-envelope layer remains fixed while the upstream source changes with the certification and runtime budget, and LECT can cache the resulting evidence without depending on the obstacle set.

C. C-space OBB Endpoint Envelopes

Axis-aligned boxes remain the global forest primitive because membership, adjacency, LECT replay, and local maintenance are simple in the dyadic joint grid. Some bridge repairs, however, are strongly correlated in joint space: a transition may move one joint up while another moves down, and the axis-aligned bounding box of that segment includes many joint combinations that never occur on the transition. For these local cases RBF can validate a typed C-space OBB tube without replacing the global forest.

We define OBBs in scaled joint coordinates rather than raw joint coordinates. Let $y = S(q - q_{\mathrm{ref}})$, where S is a diagonal joint scaling, and let

$$R_{\mathrm{obb}} = \{\, q_c + A\xi \mid \xi \in [-1,1]^m \,\}, \qquad A = S^{-1}U\,\mathrm{diag}(h),$$

with U orthonormal in the scaled metric and h the halfwidth vector. The object R_{obb} is a configuration-space region, not a workspace OBB: the forward-kinematic image of this parallelepiped is still a curved swept set. The certificate therefore comes only from endpoint/link envelopes computed over R_{obb}, not from the OBB fit or its orientation.

a) Cover certificate: The lowest-risk validation path treats the OBB as a macro-region and proves it with existing joint-box certificates. Split the local cube into tiles $[-1,1]^m = \cup_r X_r$, map each tile to joint space, and take its axis-aligned hull

$$I_r = \mathrm{bbox}_q(q_c + AX_r).$$

If every I_r passes the standard conservative link-envelope scene test, then

$$R_{\mathrm{obb}} \subseteq \bigcup_r I_r \subseteq \mathcal{C}_{\mathrm{free}}.$$

Here $\mathcal{C}_{\mathrm{free}}$ denotes the collision-free configuration set under the stated collision model. The stored OBB certificate is then a finite cover of ordinary box certificates. This path reuses the current IFK/HIFK, AABB broadphase, and SupportHull stages; its cost is the number of cover tiles needed when the OBB is highly oblique.

b) Correlated endpoint envelope: A tighter OBB-specific source keeps the local variables ξ correlated through the forward-kinematics computation. In an affine-arithmetic implementation, each joint is seeded as

$$q_j = q_{c,j} + a_j^\top \xi, \qquad \xi_i \in [-1, 1],$$

where $a_j^\top$ is the j-th row of A. Trigonometric and rigid-body operations propagate these shared symbols plus conservative nonlinear remainder symbols, producing endpoint records of the form

$$E_k(R_{\text{obb}}) = c_k + \sum_i g_{k,i}\epsilon_i \oplus [-\rho_k, \rho_k].$$

An equivalent local Taylor-zonotope form is

$$E_k(R_{\text{obb}}) = p_k(q_c) + J_k(q_c)A[-1, 1]^m \oplus [-\rho_k, \rho_k],$$

where each coordinate remainder is bounded conservatively, for example by

$$\rho_{k,\mu} \geq \frac{1}{2} \sum_{a,b} \left| \left(A^\top H_{k,\mu} A\right)_{ab} \right|,$$

with $H_{k,\mu}$ an interval or otherwise conservative Hessian bound over R_{obb}. If the Hessian or affine remainder is sampled rather than outward bounded, the OBB source is only a performance filter and the path must be accepted through the independent query-validation policy.

OBB-derived endpoint records feed the same endpoint-to-link construction as joint boxes. For a zonotope-plus-box endpoint record, the support function is

$$h_{E_k}(d) = d^\top c_k + \sum_i |d^\top g_{k,i}| + \sum_{\mu=1}^{3} |d_\mu| \rho_{k,\mu}.$$

The zero-radius link centerline support is

$$h^\circ_\ell(d) = \max\{h_{E_a}(d), h_{E_b}(d)\},$$

and the rounded-link envelope adds the capsule radius,

$$h_\ell(d) = h^\circ_\ell(d) + r_\ell \|d\|_2.$$

Coordinate-axis queries of h_ℓ provide an AABB broad-phase, while the same support record can be used by the SupportHull/GJK narrow phase. Thus a validated OBB corridor edge is just another conservative region in the typed query graph: it inherits the corridor certificate only through its stored envelope certificate and overlap witnesses, and it is kept separate from the persistent dyadic LECT tree unless promoted to a scene-local overlay.

D. Link-Envelope Representations

Once endpoint envelopes are available, the remaining design question is how to represent the swept link volume compactly for box growth, collision filtering, and optional reuse. For a capsule link, RBF does not reason about the thick link directly. It first bounds the zero-radius center segment between the two endpoint envelopes and then pads that centerline envelope by the link radius. A conservative link interval envelope $B_\ell(I)$ is any set that contains the full swept capsule over interval I. In the box-based realization used below this is written as $B_\ell(I) = B^\circ_\ell(I) \oplus \mathcal{R}(r_\ell)$, where $B^\circ_\ell(I)$ bounds the centerline sweep and the padding operator $\mathcal{R}$ becomes B_∞ for AABB envelopes.

The following representations share the same endpoint evidence but preserve different amounts of downstream geometric detail.

a) AABB envelope: AABB envelopes are the simplest option. The standard link-interval AABB construction takes the two endpoint envelopes, wraps them in one box, and pads that box by the link radius:

$$B_\ell(I) = \text{bbox}\big(E_{\ell-1}(I) \cup E_\ell(I)\big) \oplus B_\infty(r_\ell).$$

This representation can be tightened by subdividing the reference center segment into S pieces, bounding each piece separately, and retaining the union of the padded sub-envelopes. Larger S preserves more directional variation along the link while keeping reconstruction efficient once endpoint envelopes are available. The main caveat is that the subdivision must remain explicit: if all sub-envelopes are collapsed back into one global AABB, much of the gain disappears.

b) SupportHull envelope: When the endpoint envelopes are convex, their convex hull preserves directional structure that a single AABB discards. SupportHull is the compact runtime realization of that idea: instead of storing an explicit dense hull, it keeps the two endpoint AABBs and the link radius as a support-function record. For a query direction d, the support point is the better of the proximal and distal AABB support points, and collision refinement against an obstacle AABB uses a fixed-size Gilbert–Johnson–Keerthi (GJK) narrow phase with the obstacle inflated by the link radius and safety margin. This retains much of the convex- hull directional fidelity while keeping the record compact.

Thus the envelope layer is modular with respect to the upstream kinematics solver: endpoint records determine the link envelope, and the chosen representation sets the planning-time cost/tightness trade-off.

IV. Lifelong Envelope Cache Tree

The planner can materialize envelopes online; LECT is an optional replay layer for repeated builds or expensive envelope representations. It stores scene-independent endpoint/link-envelope evidence keyed by robot kinematics and joint intervals. Obstacle tests, box labels, graph connectivity, and query outcomes remain scene-local.

LECT is therefore separate from the scene-level RBF cache. The forest and corridor graph answer multiple queries in one fixed environment; LECT sits below that cache and reuses compatible kinematic evidence during builds, rebuilds, or local refills.

A. Cache Key and Split Policy

LECT is a dynamically grown binary k-dimensional (KD) tree over the joint-limit box $\mathcal{Q}$. Each node represents one joint interval and stores interval bounds, split metadata, cached endpoint evidence from which link envelopes can be reconstructed, and residency state for the

runtime. The key property is that the stored evidence is *kinematics-keyed* rather than *scene-keyed*: later scenes may replay a materialized envelope record, but they must rerun their own scene-local obstacle tests. Nodes use a heap-style breadth-first indexing convention (0 at the root and $2p+1, 2p+2$ for children), which gives a finite indexed tree with configurable maximum depth $D_{\max}$.

a) Evidence/label separation: An envelope record is replay-compatible only when the robot model, endpoint source, envelope representation, split descriptor, link radii, and active-link set match the current build. Replaying such a record imports endpoint and link-envelope evidence, not a free-space label. Every scene must still test the reconstructed link envelopes against its own obstacle set before a box can be accepted into the forest. This separation is what allows LECT to accelerate repeated geometry materialization without becoming a stale collision database.

The split policy is configurable and canonical for a given cache descriptor. The implemented schedules include round-robin, widest-root, and an affine-arithmetic volume-minimizing schedule. A build fixes one descriptor so that replay compatibility is well-defined; alternative descriptors are treated as separate cache settings. For a parent interval I, let I_d^- and I_d^+ be the two children obtained by splitting joint dimension d at its midpoint, and let $\mathcal{V}(I) := \sum_\ell \mathrm{vol}(B_\ell(I))$ be the cumulative downstream envelope volume. The volume-minimizing schedule chooses depth-wise split dimensions that minimize a worst-child envelope burden,

$$d^* = \arg\min_d \max\left\{ \sum_\ell \mathrm{vol}(B_\ell(I_d^-)), \sum_\ell \mathrm{vol}(B_\ell(I_d^+)) \right\}.$$

The resulting descriptor is depth-synchronous: all nodes at the same depth use the same scheduled dimension when that dimension is still wide enough, otherwise the runtime falls back to a valid widest dimension. Candidate dimensions may also be filtered by forward-kinematics reachability, since splitting a joint beyond the last active frame that contributes to any link endpoint cannot tighten the endpoint envelopes.

b) Sparse interval staging: The compressed runtime path does not require every intermediate heap node on a deep split path to be materialized. A materialized planning cell can instead be represented by its root copy, per-dimension target-grid interval $[\iota_d^-, \iota_d^+)$, split-count vector, and exact joint interval. This is a Patricia-style overlay on the depth-synchronous split policy: branching and terminal cells are indexed, while unvisited ancestors remain virtual. The staging record also carries the interval fingerprint and split-policy hash used by LECT's exact interval replay path. Thus evidence replay remains keyed by the robot-compatible joint interval through an exact-interval replay lookup; if no exact record is available, the same interval is live-materialized and then validated against the scene. Scene labels are still stored only in the forest or partition layer, not in LECT.

B. Runtime Use and Validation Boundary

LECT is used by local box materialization, not by the graph search itself. Given a joint interval, the cache either replays endpoint evidence or materializes it online, then derives the requested link representation. Directional records are used as refinement evidence rather than as standalone free-space certificates: the runtime first evaluates padded link AABBs, then uses SupportHull/GJK only for survivors of the AABB test. When a node is split, child envelopes may refine the parent witness, but any compact parent record remains representation-dependent and is rechecked against the current obstacle set before a box can enter the forest.

C. Storage and Warm-Build Semantics

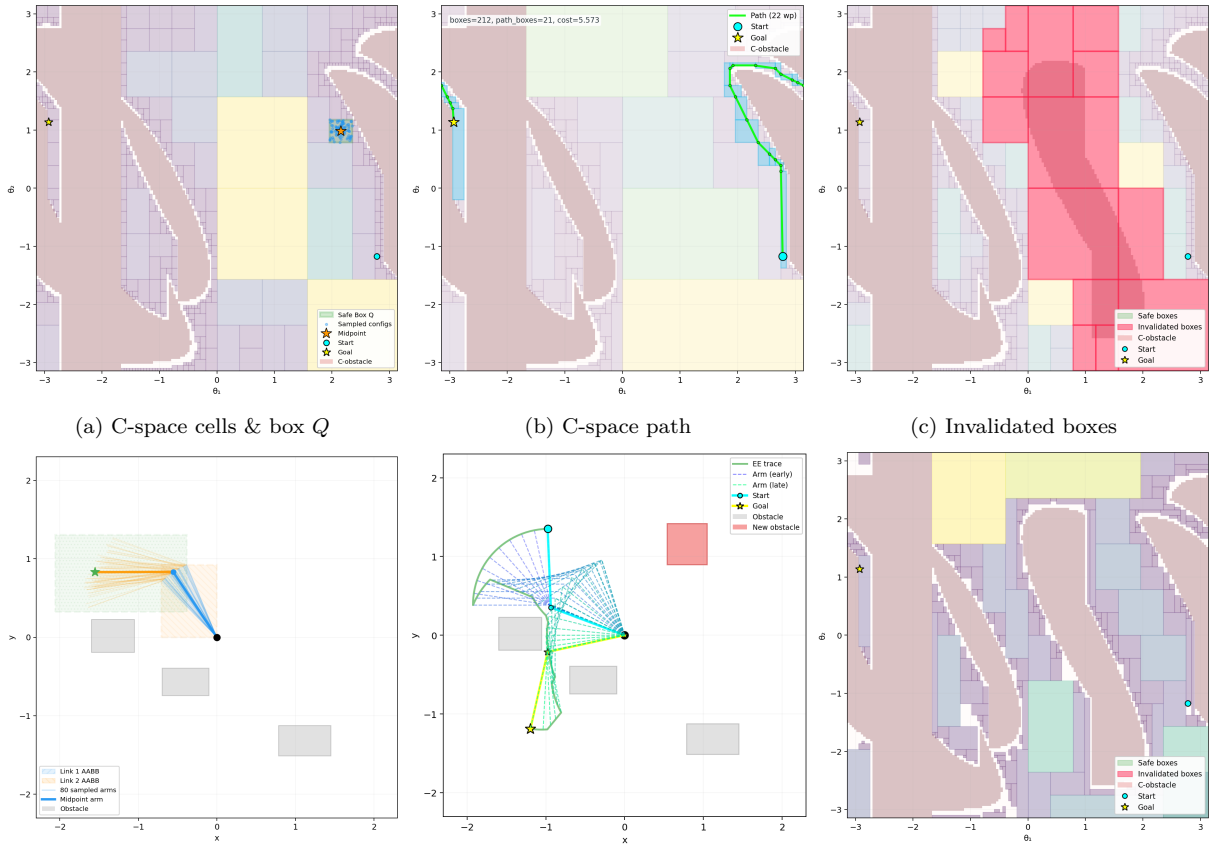
Persistence is selective. Endpoint evidence is the common reusable record, but simple IFK+AABB evidence can be lower-cost to recompute online than to reload. SupportHull records are stored only when their tighter filters are expected to amortize storage and lookup cost. Reuse accounting should therefore attribute cache hits only to compatible kinematic evidence replay and not rely on additional symmetry-transfer assumptions.

The prewarm phase follows the same separation. It materializes forward-kinematics/link-envelope evidence only on the target leaf layer and reconstructs upper layers by child-hull propagation during checkpoint. This avoids charging direct forward-kinematics at multiple depths for the same cache and keeps parent witnesses tied to the leaf records that generated them. Exact leaf evidence and propagated child-hull evidence are both treated as envelope records by the replay path; provenance does not create a separate reuse or validation mode.

In the warm-build path, a cache hit reconstructs the requested link-envelope representation from stored endpoint records and revalidates it against the current obstacles. Warm builds still regrow scene-specific boxes; persisted box labels are not imported as certificates. Obstacle checking, adjacency, scene-level query reuse, and reactions to changing $\mathcal{O}$ remain in the forest routines. Fixed-replay studies can disable writeback, while cache-growth studies use a separate writeback policy.

V. RapidBoxForest Construction

Figure 2 gives a didactic two-link view of the planning layer. The algorithm grows a scene-specific forest of C-space boxes and exports it as a typed corridor graph for repeated queries. This forest-and-graph pair is the scene-level cache: while the obstacle set is unchanged, new queries use membership lookup and graph search instead of regrowing the forest. Samples choose frontiers, the envelope oracle validates boxes, and accepted boxes extend the connected region graph. The link interval envelopes of section III and LECT of section IV provide the kinematic evidence used by this scene-level decomposition.



(a) C-space cells & box Q (b) C-space path (c) Invalidated boxes

(d) Workspace AABBs of Q (e) Workspace path (f) After regrowth

Fig. 2. Two-link didactic illustration of the RBF build, query, and obstacle-update cycle. A highlighted C-space box maps to workspace AABBs; the box corridor yields a path; obstacle insertion invalidates intersecting boxes and triggers local refill without a full rebuild. This is a low-dimensional teaching rendering of the build/query/update logic, not the robot geometry, workspace scale, or dimensionality of the reported experiments.

a) Box, cell, and domain terminology: In the planner, a *box* is an axis-aligned joint interval $B = \prod_d [l^{(d)}, u^{(d)}] \subseteq \mathcal{Q}$. The term *cell* is used when viewing the same object as an adaptive-decomposition element. A *collision domain* is a cell that has not been admitted to the forest under the scene-validation policy but is retained as a bounded refinement or update region. Thus boxes, cells, and domains share the same interval representation; the terms distinguish their role in the construction. A *terminal cell* is a leaf-sweep cell that is no longer split under the current budget or policy; if it has passed conservative scene validation, it is a *certified-free cell*. A domain may therefore be retained for later refinement or update even when it is not currently a validated forest box.

b) Validation terminology: Throughout this section, a *conservatively validated box* is a joint box whose conservative link envelopes are disjoint from the current obstacle set. These boxes support the formal corridor statement below. A *performance-mode box* is accepted by a tighter nonconservative filter and is therefore treated as a planning candidate until the recovered path passes query validation. We use *validated box* for the box admitted by the selected scene-validation policy. This policy is the box-admission rule used during forest construction: it may be the conservative envelope-disjointness test, a performance-mode filter, or the same test augmented with task-specific scene constraints. It is separate from the query-validation path, which checks recovered paths that do not inherit the conservative box-corridor certificate. We explicitly state when the stronger conservative certificate is required. Paths that leave the conservative box union through repair, smoothing, or external composition are charged where used and passed through the same query validation.

The build process has four stages. First, a seed-guided local oracle adaptively refines and classifies a C-space cell around a selected configuration. Second, the leaf-sweep mode constructs a shallow, reusable partition of the current scene and identifies conservative refinement domains. Third, repeated frontier expansion or restricted refinement grows validated cells into a connected forest, giving the method a region-valued, frontier-biased construction. Fourth, consolidation coarsens redundant boxes before the structure is exported as a query graph. The consolidation pass is part of the reusable build phase.

A. Construction Objective

Let $\mathcal{Q}$ denote the joint-limit box and let $\mathcal{O}$ denote the current obstacle set. The construction objective is to build a finite family

$$\mathcal{F} = \{B_1, \ldots, B_N\}, \qquad B_i \subseteq \mathcal{Q},$$

such that every B_i is admitted by the selected scene-validation policy, the union $\bigcup_i B_i$ covers as much query-relevant free space as possible under the available box, time, and depth budgets, and the induced adjacency graph supports efficient downstream search. In conservative mode these admitted boxes are formal free-space certificates; in performance-oriented modes they are planning regions whose recovered paths require query validation. This object is scene-specific: changing $\mathcal{O}$ changes which intervals are accepted into the forest, even though the underlying envelope evidence in LECT remains reusable.

The central structural rule is that the forest is not merely a free-space cover, but a *corridor-compatible* free-space cover. Boxes are inserted under a connected-box constraint.

Viewed as adaptive cell decomposition, this objective is intentionally workload-biased rather than globally exhaustive. The planner does not try to classify every cell in $\mathcal{Q}$. It refines and accepts cells near query-relevant frontiers, task seeds, and connector candidates, then keeps the resulting adjacency graph as the reusable planning object.

a) Connected-box rule: For boxes

$$B_i = \prod_{d=1}^{n} [l_i^{(d)}, u_i^{(d)}], \qquad B_j = \prod_{d=1}^{n} [l_j^{(d)}, u_j^{(d)}],$$

RBF treats them as connected when, in every coordinate, the two intervals either overlap or are separated by at most a small tolerance ε_c:

$$\max\{l_i^{(d)}, l_j^{(d)}\} \le \min\{u_i^{(d)}, u_j^{(d)}\} + \varepsilon_c, \qquad d = 1, \dots, n.$$

When $\varepsilon_c = 0$, this reduces to standard interval intersection or boundary touching in every dimension. A small positive tolerance absorbs the boundary offset used by grower seeds and floating-point roundoff.

This rule is deliberately weaker than requiring a full shared face. It treats overlap, touching, and tolerance-scale gaps uniformly as graph-connectivity evidence, while validity remains attached to the accepted box union itself. Only overlapping or touching box chains directly imply a contained corridor; tolerance-gap edges are query candidates that must be bridged by contained waypoints or by explicit segment validation. The output of the construction stage is therefore a forest of validated boxes whose graph connectivity is aligned with the corridor object queried later, without overstating a stricter face-sharing invariant than the grower actually maintains.

B. Local Free-Box Materialization

The local materialization procedure descends LECT from the root toward the leaf containing seed $q \in \mathcal{Q}$, materializing endpoint and link envelopes lazily along the path and returning the first admissible validated ancestor on the seed path. This is the largest canonical node on that path accepted by the scene-validation policy before further descent is needed. Tree topology and endpoint records are reused from the scene-independent cache, while scene-specific validation is performed on each grower visit. Algorithm 1 gives the pseudocode.

Algorithm 1 Local box materialization around a seed.

Input: Seed configuration q, LECT T, obstacle set $\mathcal{O}$, depth cap $D_{\max}$
Output: Validated interval I_v or failure

$v \leftarrow \text{root}(T)$; $d \leftarrow 0$; initialize current interval and kinematic state
while $d \le D_{\max}$ **do**
  Cache materialization
  materialize the endpoint and link envelopes of v if needed
  Reservation and scene validation
  if v is reserved and cannot be split further **then**
    return failure
  end if
  if the admission policy permits testing v and v passes scene validation **then**
    return interval I_v
  end if
  Refinement
  if $d = D_{\max}$ **then**
    return failure
  end if
  if v has no children **then**
    split v along the depth-synchronous split dimension of depth d
  end if
  $k \leftarrow$ split dimension of v
  $v \leftarrow$ the child of v whose interval contains q; update the current interval
  update the kinematic state from dimension k onward; $d \leftarrow d + 1$
end while
return failure

The termination behavior follows directly from the depth cap. Under $D_{\max}$, each loop iteration either returns immediately or descends one more level in the tree, so the procedure can make at most $D_{\max} + 1$ decisions. In strict conservative mode, any returned interval has passed envelope disjointness against $\mathcal{O}$; in performance-oriented modes, the returned interval is a validated planning region that remains subject to the common query-validation path.

C. Adaptive Leaf Sweep and Restricted Refinement

The fast construction path uses local free-box materialization inside an adaptive terminal cell routine. Rather than relying only on stochastic frontier proposals to discover query-relevant coarse regions, RBF starts from a shallow depth-synchronous cell set and recursively classifies only cells that remain useful under the build budget. A terminal cell is one of five types: certified-free, certified-occupied, inconclusive-domain, deferred, or covered by an already accepted box. A certified-free cell is inserted into the initial forest and its descendants are pruned. An inconclusive cell is split only when its depth, blocker report, adjacency to accepted boxes, and seed evidence suggest that further refinement can improve the reusable cover. The fixed virtual leaf sweep is recovered as a special case that forces every inconclusive cell to split until the terminal depth.

During adaptive classification, RBF also records a blocker set

$$\beta(C) \subseteq \{1, \dots, |\mathcal{O}|\}$$

for each non-free or deferred collision domain. This set is conservative update metadata: it contains obstacle groups

that were observed as blockers or were not proven free by the grouped shallow sweep. The blocker set is an over-approximate update explanation; certification of boxes entering the forest remains governed by the standard envelope validation rule.

An optional occupied predicate is used only when it can prove the whole cell is inside an obstacle. Otherwise the cell remains inconclusive or deferred.

Lemma 1 (Signed-distance occupied pruning)**.** *Let x_0 be a material point on link ℓ, let q_c be the center of cell I, and let ϕ_O be the signed distance to obstacle O in workspace. If the workspace motion of this material point over I is bounded by $\rho_{\ell,x_0}(I)$ and*

$$\phi_O(T_\ell(q_c)x_0) + \rho_{\ell,x_0}(I) + \epsilon_{\text{num}} < 0,$$

then every configuration in I places that material point inside O, so I can be recorded as a certified occupied domain for pruning.

When such a witness is unavailable, RBF does not infer occupancy; it keeps the cell as an inconclusive or deferred refinement domain.

After the shallow sweep, restricted refinement is performed only inside selected collision domains. Domains are ranked by their adjacency to already certified boxes, volume, and proximity to query-priority configurations. Candidate seeds are placed at interfaces between a collision domain and neighboring free boxes, with additional center and boundary seeds to avoid depending on a single contact point. For a domain C, every accepted refined box B must satisfy

$$B \subseteq C$$

and must be adjacent either to an existing forest box or to a box already accepted from the same domain. Thus the refinement stage can fill narrow passages revealed inside a shallow collision domain without spending its budget on unrelated regions of $\mathcal{Q}$. The connector stage may subsequently add collision-checked segment witnesses between remaining islands, but these witnesses are recorded separately from box-overlap edges.

If N_{mat} cells are materialized or replayed and N_{term} terminal cells are retained, the sweep work is $O(N_{\text{mat}}(C_{\text{env}} + C_{\text{obs}}) + C_{\text{adj}})$ and the terminal storage is $O(N_{\text{term}})$, where C_{env} is the envelope materialization or replay cost, C_{obs} is scene validation cost, and C_{adj} is the indexed adjacency update cost. The important distinction from a fixed virtual layer is that N_{mat} is driven by accepted and frontier cells, not by the full cardinality of $\mathcal{L}_{d_0:d_1}$.

D. Forest Growth

Starting from an initial seed set, the forest is grown by repeatedly invoking local free-box materialization on carefully chosen candidate configurations. Each seed first produces a root box through the same local materialization step, and these root boxes initialize one or more connected components of the forest. The main grower is an evidence-guided, frontier-biased expansion process rather than a generic random cover. Each iteration chooses a frontier component, proposes a nearby interval worth testing, and accepts the result only when it extends the forest by an admissible connected corridor segment. This keeps the build process focused on boxes that can improve query connectivity rather than on uniformly covering all of $\mathcal{Q}$.

Algorithm 2 Adaptive leaf sweep with restricted refinement.

Input: LECT T, obstacle set $\mathcal{O}$, sweep depths d_0, d_1, refinement budget N_r
Output: Validated forest $\mathcal{F}$ and collision-domain cache $\mathcal{C}$

$\mathcal{F} \leftarrow \emptyset$; $\mathcal{C} \leftarrow \emptyset$
initialize a priority frontier with start cells at depth d_0
while frontier nonempty and build budget remains **do**
  pop the highest-priority cell C
  if C is covered by an accepted box **then**
    mark C covered and continue
  end if
  materialize or replay LECT evidence for C and run scene validation
  if C is certified free **then**
    insert C into $\mathcal{F}$ and prune descendants
  else if lemma 1 certifies C occupied **then**
    record C as a certified occupied terminal domain
  else if depth$(C) = d_1$ or C is low-priority under no-good/connectivity rules **then**
    record C and its blocker set $\beta(C)$ in $\mathcal{C}$
  else
    split C by the configured split policy and push selected children
  end if
end while
build adjacency among boxes in $\mathcal{F}$
for all collision domains $C \in \mathcal{C}$ in priority order **do**
  generate interface, center, and boundary seeds inside C
  for all seed q while budget remains **do**
    materialize a validated box B around q within C using T and $\mathcal{O}$
    if $B \neq \emptyset$, $B \subseteq C$, and B is adjacent to $\mathcal{F}$ or to a prior box from C **then**
      insert B into $\mathcal{F}$ and update local adjacency
    end if
  end for
end for
return $\mathcal{F}, \mathcal{C}$

Each iteration makes two explicit heuristic decisions: it samples a target configuration q_t, then chooses a boundary seed on an existing box face that is closest to that target. The target sampler is a configurable categorical mixture over intertree roots, unexplored-volume targets, query roots, fixed anchors, and uniform roots. Build connectivity is handled primarily by overlap-compatible box insertion; segment bridges remain a secondary mechanism recorded separately from box-overlap edges. The mixture weights, depth caps, cache depth, thread count, and anchor set are planner settings rather than envelope-theory parameters.

Given a target q_t and an existing box $B = \prod_j [\ell_j, u_j]$, the grower enumerates feasible faces $f = (B, d, s)$, where d is a joint dimension and $s \in \{-1, +1\}$ is the lower or upper side. Let τ be the adjacency tolerance and $\epsilon_b = \max(\epsilon_{\text{guard}}, \tau/4)$. A face is feasible only if stepping outside it by ϵ_b remains inside the global root interval. The seed

Algorithm 3 Budgeted expansion of a connected box-region forest.

Input: LECT T, obstacle set $\mathcal{O}$, seed set $\mathcal{S}$
Budgets/settings: $N_{\max}, t_{\max}, K_{\text{face}}, K_{\text{fail}}, H_{\text{cool}}$
Output: Validated box-region forest $\mathcal{F}$
$\mathcal{F} \leftarrow \emptyset$; initialize connected components from $\mathcal{S}$
for all $q \in \mathcal{S}$ **do**
  materialize a validated box B around q using T and $\mathcal{O}$
  if $B \neq \emptyset$ **then**
    insert B into $\mathcal{F}$ as a root box
  end if
end for
while $|\mathcal{F}| < N_{\max}$ and time budget not exhausted **do**
  draw target q_t from the configured connector/query/unexplored/uniform mixture
  enumerate feasible faces $f = (B, d, s)$ of frontier boxes and compute q_f and $\sigma(f; q_t)$
  rank the K_{face} lowest-score face seeds
  choose the first ranked seed that is not deferred and not in the frontier-seed cache
  set $(q_{\text{seed}}, B_{\text{near}}) \leftarrow (q_f, B)$ for that face
  materialize a validated box B_{new} around q_{seed} using T and $\mathcal{O}$
  if $B_{\text{new}} \neq \emptyset$ and it connects to B_{near} under tolerance τ **then**
    insert B_{new} into $\mathcal{F}$ and update connected components
  else
    record the failed LECT leaf; defer it after K_{fail} failures for horizon H_{cool}
  end if
end while
return $\mathcal{F}$

associated with the face is

$$q_f[j] = \begin{cases} u_d + \epsilon_b, & j = d,\ s = +1, \\ \ell_d - \epsilon_b, & j = d,\ s = -1, \\ \text{clip}(q_t[j], \ell_j + \epsilon_b, u_j - \epsilon_b), & j \neq d, \end{cases} \tag{1}$$

with clipping to $[\ell_j, u_j]$ when a box is thinner than $2\epsilon_b$. Faces are ranked by the squared target distance

$$\sigma(f; q_t) = \|q_t - q_f\|_2^2, \tag{2}$$

and the grower retains a bounded candidate set of size K_{face} across the scanned frontier boxes. It then tries them in increasing score order, skipping a candidate if the corresponding LECT leaf is temporarily deferred or if a component-connection cache has already recorded the same boundary seed. The first remaining candidate becomes the seed for local materialization.

The candidate box is accepted only if local materialization validates it and the new box is connected to its parent under the same adjacency predicate used by the query graph: all dimensions overlap up to tolerance τ, and at least one dimension touches within tolerance or all dimensions overlap. Failures are also handled explicitly. When the LECT leaf containing a failed seed reaches the configured hard-frontier failure threshold K_{fail}, the leaf is skipped for a deferral horizon H_{cool} of subsequently accepted boxes, unless a later depth stage raises the active search depth and enables a retry. This deferral rule and the frontier-seed cache do not alter the validation boundary; they only reduce repeated calls on already failed or already covered frontier regions. The resulting method is summarized in algorithm 3.

The same construction also supports coordinated multi-goal growth. Multiple seed configurations can initialize roots in one authoritative forest, allowing the scene-level RBF cache to support a batch of related queries while sharing compatible LECT evidence across build workers and cache-warm reruns. Staged wavefront growth and task-batched growth are scheduling choices; they do not change the mathematical object produced by the planner, namely the query-ready box-region graph.

Algorithm 4 Graph-preserving merging and pruning.

Input: Validated forest $\mathcal{F}$, LECT T, obstacle set $\mathcal{O}$
Output: Consolidated validated forest $\mathcal{F}'$
$\mathcal{F}' \leftarrow \mathcal{F}$
Stage 1: validated connected/contact merging
merge exact connected pairs whose union is itself a single box
for all connected candidate pairs (B_i, B_j) in $\mathcal{F}'$ **do**
  build a candidate parent region H and score its volume expansion
  if the chosen envelope representation of H passes the scene-validation policy **then**
    replace B_i, B_j by H
  end if
end for
Stage 2: containment pruning
remove only contained boxes whose typed adjacency and segment witnesses transfer to selected covering regions
return $\mathcal{F}'$

E. Forest Consolidation

Raw growth tends to produce a forest that is finer than necessary. Consolidation reduces this representation without changing the validation rule or the typed-edge semantics used by corridor search. The planner uses two conservative operations. First, connected pairs whose union is again a box may be merged after validation. Second, more general connected candidate pairs are ranked by a volume-expansion score and accepted only if the candidate hull passes the same scene oracle. A final containment-pruning pass removes boxes that are fully covered by accepted validated regions when their graph incidence can be transferred. The consolidation logic is summarized in Algorithm 4.

Validity follows directly: every accepted merge has itself passed the scene oracle. Containment pruning removes only boxes fully covered by a selected region that inherits their typed adjacency and segment witnesses, preserving the connectivity semantics used by query search.

F. Corridor Query

After growth and consolidation, the forest is exported as a typed graph $G = (V, E_\cap \cup E_\varepsilon \cup E_s \cup E_o \cup E_\pi)$ with one vertex per validated box. Edges in $E_\cap$ connect boxes that overlap or touch in all dimensions; these edges preserve a contained box corridor only when the incident boxes were conservatively validated and the extracted path remains inside their union. If a box was admitted only by a performance-mode filter, the same graph edge is search structure and the recovered path must pass query validation. Edges in E_ε come only from tolerance-scale connected-box gaps and are treated as query candidates rather than as corridor certificates. Edges in E_s are explicit collision-checked residual segment witnesses. Edges in E_o

are typed C-space OBB corridor edges: each stores one or more validated OBB regions that cover the repaired transition while preserving joint correlation in the endpoint-envelope validator. Edges in E_π are conservative portal edges: each stores a compressed edge between two global boxes together with a finite hidden chain of conservatively validated internal boxes. The chain is not materialized as global graph vertices, but it is expanded when the edge is used by query path extraction. Because typed connectivity is maintained during growth and consolidation, online querying reduces to membership lookup and typed graph search rather than post-hoc graph reconstruction. In returned-path accounting, E_ε and E_s edges always require final query validation. $E_\cap$, validated E_o, and expanded E_π edges inherit the corridor theorem only when the corresponding boxes, OBB regions, or hidden chain are conservative and the extracted path stays inside their union.

If disconnected adjacency islands remain after growth, an optional connector stage attempts targeted bidirectional sampling bridges between nearest frontier boxes of different components. A successful bridge is first a path witness rather than a claim that the two boxes overlap; in the reported profile it is accepted only after conversion to a conservative E_o OBB corridor edge. If OBB conversion is disabled, the same interface can retain an E_s residual segment witness. This distinction is important for both the RBF query layer and the GCS negative controls: a segment witness is not an overlap relation, whereas a GCS edge over convex regions requires feasible overlap of the regions themselves.

The OBB conversion is local to query-critical transitions and does not replace the global AABB forest. For a bridge window, the implementation forms a thin C-space OBB tube in scaled joint coordinates with its primary axis aligned to the bridge tangent; transverse growth is deliberately small. The tube is accepted only when its endpoint/link sweep is certified by the same rounded-link scene checker, using the correlated OBB support validator when available and a sampled-support Lipschitz fallback for thin tubes. Thus an accepted E_o edge represents a reusable volumetric transition certificate, whereas a failed OBB conversion leaves the bridge as an ordinary residual segment witness and is reported separately.

This pair $(\mathcal{F}, G)$ is the planning object produced by the build stage. It is also the scene-level RBF cache for subsequent queries in the same environment. Given start and goal configurations q_s, q_g, the planner associates them with start-side and goal-side boxes in $\mathcal{F}$ and then searches G for a connecting box sequence. The online query problem is reduced to box-corridor retrieval inside an already constructed box-region representation.

A shortest path on G yields a candidate box sequence between the two query endpoints. When the recovered geometric path stays inside the union of conservatively validated boxes, it is certified collision-free by theorem 1. Paths that use E_ε or residual E_s edges, performance-mode boxes, or post-processing steps that leave the conservative box/OBB union are treated through the configured query-validation path, and the query interface records these cases through query-result counters. The box-corridor statement below applies to paths contained in that union. OBB edges in E_o and portal edges in E_π are nonsegment edges: they inherit the conservative box-corridor certificate only through their stored OBB certificates or internal box chains.

For an $E_\cap$-only graph path, this containment condition has a direct constructive interpretation. Each vertex box is convex, and each $E_\cap$ edge means that consecutive boxes overlap or touch. Therefore a continuous piecewise-linear curve can be chosen by selecting one point in each nonempty pairwise intersection and connecting these points inside the corresponding boxes. If q_s and q_g lie in the first and last boxes, respectively, the same convexity argument connects them to the first and last intersection points. The resulting curve is contained in the union of the boxes on the graph path. This is a containment certificate, not a positive-clearance certificate: touching boxes may meet at a lower-dimensional set, and clearance depends on the underlying envelope-obstacle separation or on the configured query-validation test. This construction does not apply to E_ε or E_s edges, whose semantics are candidate gap closure and explicit segment witnessing, respectively.

Corollary 2 (Conservative portal expansion)**.** *Let $(B_u, B_v) \in E_\pi$ store an internal chain $B_u, B_1, \ldots, B_m, B_v$ such that every internal box is conservatively validated and every consecutive pair overlaps or touches. Replacing the portal edge by this chain recovers an $E_\cap$-only box-corridor segment. Therefore any query path using such portal edges is covered by theorem 1 after all portals are expanded.*

Theorem 1 (Conditional conservative box-corridor path)**.** *Let $\mathcal{F}$ be a forest whose boxes have each passed conservative link-envelope validation against obstacle set $\mathcal{O}$. If a continuous path $\gamma : [0,1] \to \mathcal{Q}$ satisfies $\gamma([0,1]) \subseteq \bigcup_{B\in\mathcal{F}} B$, then every configuration on γ is collision-free with respect to the collision model in assumption 1.*

Proof. For any $s \in [0,1]$, the containment assumption gives a conservatively validated box $B \in \mathcal{F}$ with $\gamma(s) \in B$. By construction, validation of B means every conservative link envelope for B is disjoint from $\mathcal{O}$. Because each such envelope contains the link geometry for every configuration inside B (corollary 1), the robot geometry at $\gamma(s)$ is also disjoint from $\mathcal{O}$. Since s was arbitrary, the whole path is collision-free. □

Proposition 2 (Resolution-limit sufficient conditions)**.** *Assume that the obstacle set is closed, a reference path $\gamma : [0,1] \to \mathcal{Q}$ has positive clearance $\delta > 0$ under the collision model of assumption 1, joint boxes are exhaustively refined with diameter tending to zero, and the conservative endpoint/link-envelope over-approximation converges to the exact link sweep as box diameter tends to zero. Then there exists a finite chain of sufficiently small conservatively validated boxes whose union contains $\gamma([0,1])$.*

Proof. Positive clearance and closed obstacles imply an open collision-free tube around the compact set $\gamma([0,1])$. By envelope convergence, every point on the path has a small enough joint-box neighborhood whose conservative link envelopes remain inside that tube. These neighborhoods form an open cover of the compact path image, so a finite subcover exists. Exhaustive refinement with vanishing box diameter eventually generates boxes contained in that subcover; ordering them along the continuous path gives the required finite chain. □

a) Reporting protocol: Theorem 1 is a conditional statement: it covers path portions contained in a conservatively validated box union. Post-processing inherits this certificate only when it preserves that containment condition. The result is therefore a *soundness* statement for the returned box-contained corridor, not an unconditional claim that a finite-budget forest covers every feasible passage in $\mathcal{Q}$. Coverage is controlled by the selected anchor set, subdivision depth, refinement budget, and update policy. In the conservative mode, any path retained inside the accepted conservative box union inherits the certificate above; paths that require uncovered regions must wait for additional refinement, a larger forest budget, or an explicit query-validation path.

b) Completeness and failure modes: Proposition 2 is not a guarantee for the finite profiles used in the experiments. The implemented planner has depth, time, memory, and box-count caps; its frontiers are workload-biased; and its axis-aligned boxes can reject feasible narrow passages when envelope over-approximation, obstacle proximity, or split scheduling leaves no accepted cell under the available budget. Tolerance-gap edges, residual segment witnesses, OBB corridor repair, and smoothing can recover some of these cases in the reported system, but they do not extend the conservative box-corridor theorem unless the resulting path is again contained in a conservative box/OBB union. Otherwise the returned path is counted only after independent query validation.

c) Reference implementation contract: For reproduction, every reported RBF row fixes the same stage order: LECT evidence is constructed or replayed from a key containing the robot model, endpoint source, envelope representation, link radii, active-link mask, split descriptor, and depth descriptor; the scene build then validates boxes and overlap/contact edges without receiving query labels or inserting offline shortcut/ordinary segment edges; online queries anchor start/goal states, search typed graph edges, and invoke bridge repair only when graph search cannot return an audited path; finally, every returned path is strict-audited before success is counted. Cache hits may replace endpoint-envelope materialization but never bypass scene validation. Build time, per-query online latency, final simplification, strict audit, and external-cache precomputation are separate timing buckets. The reported depths, bridge limits, envelope representations, and cache descriptors are engineering profile choices for this contract, not additional assumptions in the corridor-certificate theorem.

Algorithm 5 Obstacle deletion promotion and insertion refill.

Input: Forest $\mathcal{F}$, typed graph G, collision-domain cache $\mathcal{C}$, edit (Δ^-, Δ^+)
Output: Updated forest, typed graph, and collision-domain cache

if $\Delta^- \neq \emptyset$ **then**
  for all $C \in \mathcal{C}$ **do**
    remove deleted obstacle indices from $\beta(C)$ and remap surviving indices
    if $\beta(C) = \emptyset$ **then**
      promote C to $\mathcal{F}$ only if containment, adjacency, and edited-scene validation pass
    end if
  end for
  incrementally connect promoted boxes to overlapping or touching forest boxes
end if
if $\Delta^+ \neq \emptyset$ **then**
  $\mathcal{I} \leftarrow \{B \in \mathcal{F} : B$ conflicts with some obstacle in $\Delta^+\}$
  remove $\mathcal{I}$ and typed graph/segment witnesses that fail edited-scene validation
  for all $B \in \mathcal{I}$ **do**
    set the local sweep depth, optionally capped relative to depth(B)
    sweep leaves inside B to that local depth
    insert swept leaves that pass edited-scene validation and are adjacent to the surviving forest
    cache still-blocked leaves with blockers in Δ^+
  end for
  incrementally connect newly inserted boxes and revalidate surviving segment edges
end if
if the requested query is still disconnected **then**
  optionally run endpoint recovery and collision-checked segment recovery
end if
return updated $(\mathcal{F}, G, \mathcal{C})$

G. Dynamic Obstacle Updates

The explicit forest representation also supports local typed-graph maintenance under obstacle edits. The update procedure assumes fixed robot kinematics and a fixed LECT topology/evidence cache; only the obstacle set and scene-dependent forest labels and typed edges change. The key distinction is asymmetric: removing obstacles can expose previously blocked cached domains, whereas inserting obstacles may invalidate boxes and segment witnesses that were previously accepted.

The update invariant is local but explicit: every box retained or inserted after an edit must pass the edited-scene validation policy, every $E_\cap$ edge must still correspond to overlap or contact of its endpoint boxes, every E_ε edge remains only a tolerance-gap query candidate, every E_s edge must keep a valid segment witness in the edited scene, and every E_π edge must retain a conservatively validated internal box chain or be removed. The update procedure is therefore a maintenance rule for the represented forest and its typed graph, not a claim of complete rediscovery of all newly available free-space regions after an edit.

For obstacle deletion, the blocker sets $\beta(C)$ recorded during the leaf-sweep stage provide a conservative candidate-selection rule for the information that was actually cached. When obstacle indices R are removed,

each cached collision domain C updates its blocker set to $\beta(C) \setminus R$, with the remaining obstacle indices remapped to the edited scene. If this set becomes empty, the domain becomes eligible for promotion. Promotion either reuses sufficient cached disjointness evidence for the edited obstacle set or reruns the standard scene check on the reconstructed envelope record. It is also filtered by containment and adjacency checks: contained candidates are discarded, disconnected candidates remain cached, and accepted candidates are inserted into the forest and connected incrementally. Thus deletion updates use the recorded blocker cache to focus validation effort and are limited to domains represented in that cache. Discovering additional newly free regions is left to a subsequent build or refinement stage.

Obstacle insertion uses the complementary path. Each added obstacle is first tested against the current live boxes, optionally restricted by the spatial dirty region. Boxes that collide with the added-obstacle checker are removed from the forest and released from the oracle reservation map; raw boxes are checked and removed by the same edited-scene logic. Segment edges whose endpoints disappear or whose waypoint witness fails the edited-scene validation are removed as well. Each invalidated box then becomes a bounded refill domain. The insertion routine performs a local leaf sweep inside this domain up to the configured insertion depth, optionally capped relative to the invalidated box depth. Leaves that pass the scene-validation policy are reinserted; leaves that remain blocked or hit the depth cap are cached with a blocker set containing the new obstacle index. The insertion update is therefore a local invalidation-and-refill operation over the typed graph: invalidated boxes are removed, local leaf-sweep refill proposes replacement boxes, affected segment witnesses are revalidated, and typed connectivity is repaired incrementally. If the edited graph still cannot answer the query, the query pipeline may use its configured repair or segment-edge mechanisms.

If no contained conservative corridor is recovered, the box-corridor certificate does not apply; repaired, post-processed, or segment-recovered paths are accepted only through query validation.

VI. Experiments

The experiments evaluate RBF as a reusable manipulation-planning system. The Shelf+IIWA profile study uses the KUKA LBR iiwa shelf-workcell benchmark and selects the registered RBF configuration; cross-algorithm and random-scene rows inherit its validation/simplification policy unless a row sweeps a budget. Dynamic-update rows report leaf-sweep scene-cache maintenance only; random scenes and queries are saved before planner execution.

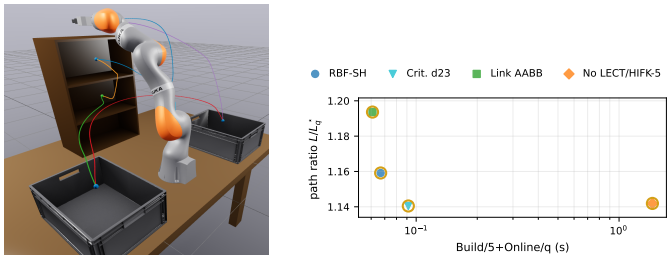
a) Protocol: Wall-clock timings were collected on an Ubuntu 22.04 x86-64 workstation with an Intel Core i5-12600KF central processing unit and 31 GiB memory. Generated tables and figures are tied to a machine-readable provenance bundle that records source summaries, component manifests, checksums, and placeholder status; the paper-generation workflow rejects missing required tables rather than silently carrying placeholder rows. Runs use one experiment process and a fixed runner/thread policy recorded in the manifest; Open Motion Planning Library (OMPL) rows do not reuse planner state and expose planner-dependent parallelism. PRM and the IRIS-NP/GCS convex-region baseline report reusable builds, RRTConnect/BIT* are one-shot rows, and all timings are wall-clock accounting comparisons rather than equal-processor-time, equal-thread, or equal-tuning dominance claims. Planner success means that the returned path passes the fixed-resolution query-validation check. Per-query online latency (Online/q) is measured from one-query reruns or archived records that explicitly store per-query solve aggregates; if an artifact also stores a reusable-batch total, that scene-level total is not used as single-query latency. Online/q excludes final simplification and strict audit. Where per-query audit time (Audit/q) is shown, audit-complete return time is Online/q+0.010 s+Audit/q; otherwise the table is online-latency context only. Numeric distributions are reported as median $[Q_1, Q_3]$, and main path ratios use success-only audited references: query-level references when saved traces are available, scenario-level imported references only where captions say so. Repaired, segment-witness, or performance-mode paths are counted only after query validation succeeds. The archived shelf diagnostics do not contain a complete waypoint-level conservative containment trace; their segment-free and repair counters are usage traces, not conservative-only or repair-free counterfactuals. The main tables are therefore final-validation summaries, not conservative-only, repair-free, or sensitivity studies. Unless stated otherwise, reported runs validate active rounded links against workspace obstacles; self-collision, attached objects, grippers, and task-specific forbidden regions require added scene checks before they inherit the certificate.

b) Cache and profile boundary: Warm-cache rows charge scene build/query work plus compatible LECT lookup and evidence reads; they exclude snapshot creation/load, memory-after-load, and full precompute amortization. The label d23 denotes the depth-23 LECT evidence snapshot used by the registered Shelf warm-replay row. That snapshot has 16,777,215 endpoint-prewarm records, 13,686,872,489 B (12.75 GiB), and 958.079 s prewarm time. If the only credited benefit were the measured 6.978–0.168 s live-vs-warm build gap, this prewarm would need about 141 five-query Shelf builds (∼705 queries) to amortize; snapshot load time and resident memory were not recorded. Table VII reports read-stage costs on a smaller Universal Robots UR5 SupportHull snapshot. Thus d23 rows are warm-replay profiles, not from-scratch planner costs or full cache-precompute break-even claims; the No-external-LECT row is a live-materialization comparison, not a pure cold d23 rerun. Cache reuse requires matching robot model, endpoint source, link representation, split/depth descriptor, radii, and active-link set.

TABLE II
Shelf+IIWA full-success RBF registered point and mechanism rows.

Case	Build	Online/q	Amort. 5q	$L/L^\star$
RBF-SH d23	0.166 [0.163, 0.170]	0.033 [0.032, 0.035]	0.067 [0.065, 0.067]	1.16 [1.08, 1.38]
Critical sample d23	0.282 [0.280, 0.285]	0.035 [0.033, 0.036]	0.091 [0.090, 0.093]	1.14 [1.08, 1.38]
Link AABB	0.141 [0.139, 0.143]	0.032 [0.031, 0.034]	0.061 [0.059, 0.064]	1.19 [1.08, 1.47]
No LECT/HIFK-5	7.006 [6.947, 7.068]	0.064 [0.061, 0.068]	1.460 [1.456, 1.473]	1.14 [1.08, 1.39]
SupportHull only	0.349 [0.287, 0.363]	0.062 [0.057, 0.068]	0.131 [0.117, 0.141]	1.16 [1.08, 1.38]

Notes: Median [Q1, Q3], s. Online/q excludes final simplification/audit; Amort. 5q = Build/5 + Online/q. SH=SupportHull; d23=depth-23 LECT; $L/L^\star$ uses the query-level strict-audited reference. d23 rows replay matching endpoint-source caches excluding prewarm/load/RSS (16,777,215 rec., 12.75 GiB, 958.079 s); No LECT/HIFK-5 materializes live.



(a) (b)

Fig. 3. Shelf+IIWA scene context and RBF mechanism study. (a) Canonical Marcucci Shelf+IIWA scene used by the fixed AS–TS–CS–LB–RB–AS saved query-label cycle. (b) Full-success points only; d23 replay excludes prewarm/load/RSS; gold rings mark Table II rows.

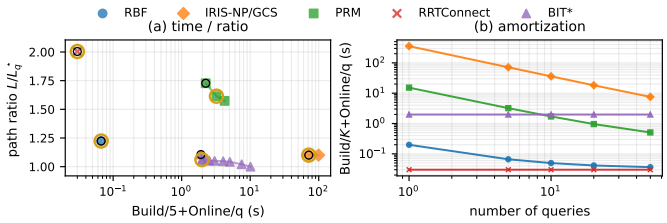


Fig. 4. Shelf+IIWA cross-algorithm trade-off. Black rings mark first full-success points; the plot uses Table III online-only timing and reusable-build terms where applicable.

TABLE III
Shelf+IIWA strict-audited per-query cross-algorithm comparison.

	RBF b100 Build 0.166 s		**IRIS-NP/GCS** Build 355.425 s		**PRM** Build 20.040 s		**RRTConnect** Build 0.000 s		**BIT*** Build 0.000 s	
Query	T (s)	$L/L^\star$	T (s)	$L/L^\star$	T (s)	$L/L^\star$	T (s)	$L/L^\star$	T (s)	$L/L^\star$
AS→TS	0.081 [0.078, 0.087]	1.39 [1.34, 1.48]	0.449 [0.449, 0.449]	1.44 [1.44, 1.44]	0.198 [0.147, 0.248]	1.61 [1.42, 1.73]	0.034 [0.024, 0.052]	2.27 [2.05, 2.72]	4.004 [4.002, 4.004]	1.06 [1.02, 1.08]
TS→CS	0.163 [0.158, 0.170]	1.12 [1.09, 1.14]	0.269 [0.269, 0.269]	1.26 [1.26, 1.26]	0.173 [0.118, 0.312]	2.99 [2.11, 3.22]	0.054 [0.021, 0.089]	3.98 [2.80, 5.09]	10.138 [10.004, 10.386]	1.09 [1.04, 1.11]
CS→LB	0.109 [0.104, 0.118]	1.47 [1.32, 1.82]	0.381 [0.381, 0.381]	1.07 [1.07, 1.07]	0.136 [0.120, 0.195]	1.49 [1.39, 2.09]	0.032 [0.027, 0.070]	2.20 [1.75, 2.51]	7.504 [7.504, 7.725]	1.03 [1.01, 1.06]
LB→RB	0.132 [0.128, 0.141]	1.15 [1.07, 1.25]	0.786 [0.786, 0.786]	1.07 [1.07, 1.07]	0.164 [0.091, 0.399]	1.18 [1.09, 1.27]	0.001 [0.001, 0.002]	1.29 [1.27, 1.32]	1.654 [1.654, 1.654]	1.02 [1.02, 1.03]
RB→AS	0.143 [0.138, 0.150]	1.04 [1.03, 1.05]	0.527 [0.527, 0.527]	1.01 [1.01, 1.01]	0.252 [0.096, 0.295]	1.23 [1.17, 1.24]	0.001 [0.001, 0.001]	1.07 [1.03, 1.10]	0.204 [0.204, 0.204]	1.02 [1.01, 1.03]

Notes: Median [Q1, Q3], s. T=Online/q excluding final simplification/audit; $L/L^\star$ uses the query-level strict-audited reference. b100=100-box RBF; IRIS-NP precedes GCS. RBF build excludes d23 prewarm/load/RSS; PRM build is the median selected roadmap checkpoint; RRTConnect/BIT* have no reusable build; PRM/BIT* select the fastest per-query checkpoint within 1.01× that query's best strict-audited mean path; no Audit/q.

Unless stated otherwise, RBF uses the Shelf-selected b100 profile, where b100 denotes the registered 100-box Shelf operating point, together with d23 read-only evidence, AABB broadphase plus SupportHull/GJK, query repair, and 0.01 rad audit spacing. Accepted bridge witnesses in the reported profile are converted into typed C-space OBB corridor edges using the correlated endpoint-envelope validator, rather than being credited as ordinary collision-checked segment edges. Random-scene rows use robot-local LECT caches and random query-independent anchors rather than Shelf-specific anchors. Portal-transition variants remain disabled in the main profile. The fixed engineering profile is intentionally reported as prose and in the artifact manifest rather than as another main table: the manuscript-facing choices are b100 for the Shelf registered build, d23 warm evidence for that row, the fixed audit/simplification policy, and saved ten-query-per-scene random catalogs. Lower-level bridge, anchoring, split, and refill constants are recorded in the artifact manifest. Unless a row name explicitly changes one of these entries, the main tables hold the profile fixed; named ablations should be read as mechanism checks around this operating point rather than as an exhaustive parameter-sensitivity sweep.

A. Shelf+IIWA Registered Profile Study

Table II and Fig. 3 show the Shelf+IIWA b100 registered profile and mechanism rows over seeds 0–7 and the fixed Marcucci AS–TS–CS–LB–RB–AS saved query-label cycle, used as deterministic labels rather than random coverage. This shelf cycle is a mechanism and profile-selection workload, not a claim about the full start-goal distribution; the saved random catalog below carries that broader distribution check. The table reports offline build time, per-query online latency, build amortized over the five shelf queries, and path ratio; final simplification and audit time are excluded from Online/q. Critical-sample rows are included only as endpoint-source checks: they use the same planner and strict audit, but the endpoint approximation is not a conservative endpoint certificate.

The selected RBF-SH d23 row, where SH denotes SupportHull link envelopes, builds the reusable scene structure in 0.168 [0.167, 0.170] s and reaches a five-query amortized time of 0.049 [0.048, 0.049] s after replaying the warm

TABLE IV
Saved-catalog random-scene selected trade-off context points.

Scenario	RBF			PRM			RRTConnect		BIT*	
	Build	Online/q	$L/L_q^\star$	Build	Online/q	$L/L_q^\star$	Online/q	$L/L_q^\star$	Online/q	$L/L_q^\star$
IIWA-Medium	0.075 [0.058, 0.083]	0.009 [0.007, 0.011]	1.00 [1.00, 1.03]	3.567 [3.007, 3.933]	0.038 [0.014, 0.070]	1.03 [1.02, 1.12]	0.002 [0.001, 0.002]	1.02 [1.01, 1.05]	0.107 [0.087, 0.154]	1.00 [1.00, 1.01]
IIWA-Hard	0.097 [0.086, 0.105]	0.014 [0.011, 0.015]	1.00 [1.00, 1.03]	4.175 [3.408, 5.637]	0.048 [0.021, 0.102]	1.03 [1.02, 1.08]	0.002 [0.002, 0.003]	1.03 [1.00, 1.07]	0.129 [0.097, 0.186]	1.00 [1.00, 1.01]
UR5-Medium	0.262 [0.254, 0.270]	0.075 [0.020, 0.086]	1.05 [1.00, 1.14]	3.124 [2.966, 3.418]	0.323 [0.085, 0.426]	1.02 [1.00, 1.12]	0.023 [0.002, 0.049]	1.23 [1.06, 1.54]	0.134 [0.101, 0.167]	1.01 [1.00, 1.08]
UR5-Hard	0.276 [0.264, 0.284]	0.083 [0.043, 0.108]	1.05 [1.01, 1.14]	2.488 [2.325, 3.048]	0.419 [0.129, 0.437]	1.06 [1.01, 1.19]	0.048 [0.008, 0.090]	1.36 [1.14, 1.58]	0.154 [0.115, 0.171]	1.03 [1.00, 1.13]
Panda-Medium	0.250 [0.218, 0.265]	0.028 [0.013, 0.039]	1.05 [1.02, 1.09]	3.431 [3.364, 4.180]	0.066 [0.038, 0.159]	1.06 [1.03, 1.08]	0.005 [0.002, 0.011]	1.22 [1.12, 1.35]	0.116 [0.092, 0.152]	1.00 [1.00, 1.00]
Panda-Hard	0.224 [0.221, 0.237]	0.043 [0.021, 0.053]	1.05 [1.01, 1.16]	2.337 [1.627, 3.978]	0.054 [0.034, 0.126]	1.10 [1.05, 1.16]	0.056 [0.019, 0.136]	1.25 [1.14, 1.37]	0.164 [0.123, 0.214]	1.01 [1.00, 1.09]

Notes: Median [Q1, Q3], s. Online/q excludes final simplification/audit; RBF/PRM include Build, RRTConnect/BIT* are online only. Cells are selected full-success checkpoints; $L/L_q^\star$ uses saved query-label references; no Audit/q.

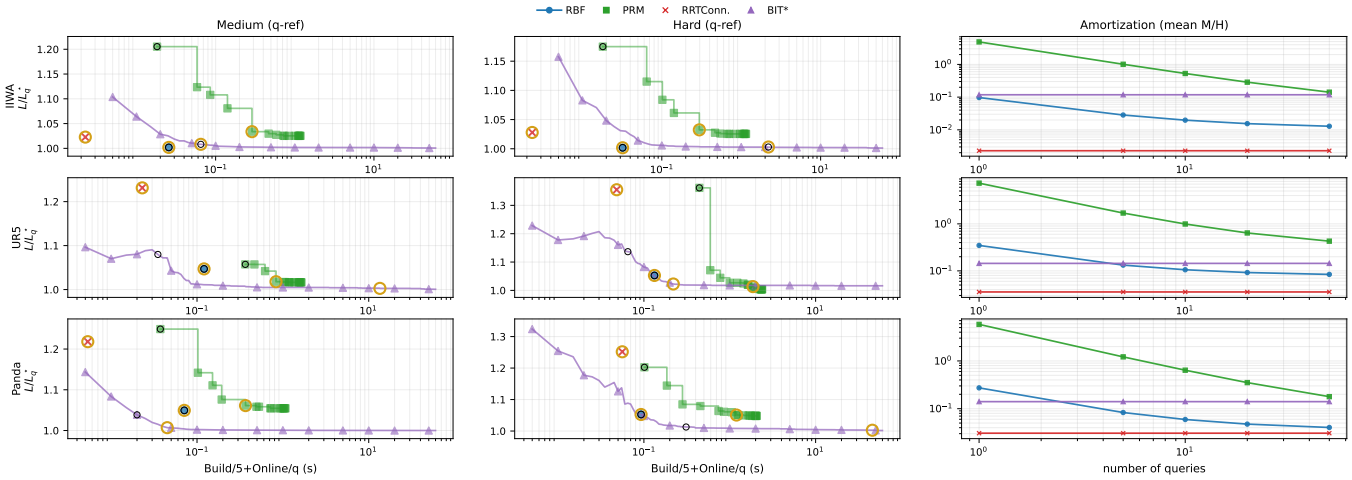


Fig. 5. Saved-catalog random-scene trade-off. Left/middle: medium/hard Build/5+Online/q with query-level path references; right: Build/K+Online/q amortization averaged over medium/hard selected rows. Black rings mark first full-catalog-success checkpoints. Gold rings advance from the black rings only while the next checkpoint improves path ratio by at least 1%, with time used only as a tie-breaker. PRM is plotted as measured cumulative roadmap checkpoints and therefore may remain improving at the largest PRM budget; BIT* is plotted from recorded anytime checkpoints, exposing its early quality drop and later stall. Partial-success checkpoints outside the table-selected operating points are shown only to visualize the envelopes.

d23 evidence snapshot. The No external LECT/HIFK-5 row raises build time to 6.978 [6.873, 7.113] s because it charges live endpoint/evidence materialization. Thus the speed gap is cache accounting, not free computation: the warm-cache row excludes the one-time 16,777,215-record endpoint-cache prewarm recorded at 958.079 s and 13,686,872,489 B (12.75 GiB) on disk, which Table II reports explicitly. In the current OBB-corridor profile, all 40 Shelf returns have zero residual ordinary-segment fraction; the typed bridge edges are represented by validated OBB corridor regions. The Link AABB and SupportHull-without-broadphase rows keep the same external evidence where possible and isolate downstream representation costs: Link AABB preserves the same selected path ratio at slightly higher online cost, while skipping the AABB broadphase roughly doubles the amortized cost. These rows are therefore mechanism checks around the registered operating point, not claims that every row changes only one factor under identical cache state.

The same shelf cycle was rerun as a validation-boundary diagnostic: the registered full profile returned 40/40 audited paths, but success dropped to 0/40 when endpoint-to-endpoint query bridge, adaptive repair, final collision shortcut, and final RRT simplification were disabled; adding endpoint-anchor and runner-exposed connector/segment suppression remained 0/40. This is a repair-dependence diagnostic rather than a conservative-only proof: the main shelf rows are final-validated repair results whose ordinary segment witnesses have been replaced, when accepted, by conservative OBB corridor certificates.

B. Shelf Cross-Algorithm Context

The Shelf+IIWA cross-algorithm study uses the same five queries and 0.01 rad final audit for RBF, the IRIS-NP/GCS convex-region baseline, PRM, RRTConnect, and BIT*. The term IRIS-NP denotes the nonlinear-programming IRIS variant used to build convex regions before GCS optimization. RBF uses the Experiment 4 b100 profile and independent one-query Online/q reruns; PRM reports one reusable five-query roadmap; IRIS-NP/GCS reports its saved convex-region build; RRTConnect and BIT* solve each query without a reusable build term. The imported IRIS-NP/GCS row is checked for matching scene, query names, source hashes, and audit spacing.

TABLE V
Endpoint envelope source study.

Width	Source	Vol.	Time (μs)	Neg. gap
0.02	IFK-AA	0.000045	5.48	0.000000
0.02	HIFK-3	0.000042	25.94	0.000000
0.02	HIFK-5	0.000042	70.40	0.000000
0.02	Critical sample	0.000041	8.68	-0.000048
0.02	Analytical	0.000041	6429.96	0.000000
0.02	MC	0.000026	2251.74	-0.006053
0.10	IFK-AA	0.007709	6.45	0.000000
0.10	HIFK-3	0.006187	29.16	0.000000
0.10	HIFK-5	0.005748	73.82	0.000000
0.10	Critical sample	0.005085	10.03	-0.001174
0.10	Analytical	0.005086	6602.25	0.000000
0.10	MC	0.003739	11233.26	-0.032677
0.20	IFK-AA	0.095828	6.58	0.000000
0.20	HIFK-3	0.063170	27.21	0.000000
0.20	HIFK-5	0.054478	73.15	0.000000
0.20	Critical sample	0.041680	11.56	-0.004527
0.20	Analytical	0.041748	6800.10	0.000000
0.20	MC	0.032592	22352.95	-0.065574
0.30	IFK-AA	0.501814	6.81	0.000000
0.30	HIFK-3	0.268384	29.27	0.000000
0.30	HIFK-5	0.214676	75.92	0.000000
0.30	Critical sample	0.141227	12.69	-0.010770
0.30	Analytical	0.142112	6927.12	0.000000
0.30	MC	0.112648	33813.58	-0.088868
0.50	IFK-AA	5.602815	7.35	0.000000
0.50	HIFK-3	2.028027	29.38	0.000000
0.50	HIFK-5	1.347150	72.32	0.000000
0.50	Critical sample	0.639900	16.33	-0.030972
0.50	Analytical	0.650907	7096.01	-0.004492
0.50	MC	0.534496	56835.57	-0.123531

Notes: IFK/HIFK are certificate-backed. Critical sample, Analytical, and Monte Carlo (MC) are diagnostics only; Neg. gap uses the sampled-union diagnostic.

TABLE VI
Link envelopes; Env. build, Coll. obstacle test.

Width	Envelope	Splits	Vol.	Env. (μs)	Coll. (μs)
0.02	Link AABB	1	0.013	0.110	0.300
0.02	SupportHull	1	0.000	0.166	0.309
0.05	Link AABB	1	0.020	0.139	0.310
0.05	SupportHull	1	0.003	0.177	0.306
0.10	Link AABB	1	0.038	0.141	0.297
0.10	SupportHull	1	0.014	0.188	0.305
0.20	Link AABB	1	0.147	0.136	0.305
0.20	SupportHull	1	0.105	0.171	0.311
0.30	Link AABB	1	0.502	0.137	0.305
0.30	SupportHull	1	0.440	0.175	4.823
0.50	Link AABB	1	4.514	0.143	0.304
0.50	SupportHull	1	4.448	0.184	10.950

TABLE VII
LECT warm-cache cost (Universal Robots UR5 SupportHull snapshot; 2,097,151 records, 1.67 GiB).

Operation	Ops	Avg. (μs)
Open snapshot	1	86.219
Node box	20,000	1.084
Exact lookup	20,000	0.849
Endpoint lookup	20,000	1.860
Endpoint lookup (hot)	20,000	0.577
Range query	20,000	22.596
Evidence read	20,000	1.262
Evidence read (hot)	20,000	0.132

TABLE VIII
Adaptive leaf-sweep maintenance only, not end-to-end replanning.

Operation	Time (ms)	Speedup
Warm build (10 obstacles)	210.35 [208.64, 212.29]	–
Insert to 15 obstacles	0.04 [0.04, 0.05]	4295.7–5155.2×
Remove from 15 obstacles	9.11 [9.02, 9.23]	22.8–23.2×
Warm build (15 obstacles)	210.21 [209.45, 210.75]	–

Notes: Median [Q1, Q3]. Maintenance only; Speedup = 15-obstacle warm build / update time.

BIT* is reported from recorded per-query checkpoint traces; the table post-hoc selects the observed checkpoint return time for a quality–time context point, not a deployable hindsight-free online stopping rule. Thus the comparison is an online-latency and amortization context for a reusable region graph, not an exhaustive-tuning contest. Under this accounting, RBF's reported warm-cache reusable build is 0.168 s excluding d23 prewarm/load/RSS, versus PRM's 15.032 s roadmap build and the 355.425 s IRIS-NP/GCS contextual build; one-shot planners can still be faster on easy individual queries.

C. Random Multi-Robot Scenes

The random-scene study uses a saved ten-query-per-scene prefix catalog generated before planner execution and reused by every method. Medium and hard scenes share ten native-joint query pairs per robot/seed, with hard extending medium; direct-line trivial cases are rejected by strict reference probes. The reported set covers KUKA LBR iiwa, Universal Robots UR5, and Franka Emika Panda over medium/hard difficulties and ten saved seeds, giving 600 saved query evaluations per planner method.

Table IV and Fig. 5 summarize the selected random-scene trade-off context points. RBF builds one query-agnostic forest per scene and reuses it for ten queries. Random rows do not reuse Shelf anchors or the Shelf d23 cache; they use robot-local LECT caches and robot-local coverage profiles. Each RBF entry is the registered full-success OBB-corridor profile selected by the paper-generation manifest: after query repair finds an RRT witness, the witness is accepted only if it can be converted into one or more certified C-space OBB corridor regions. The reported random-scene RBF rows therefore have zero residual ordinary-segment fraction. It minimizes Online/q within that registered set before ten-query amortized time and path length; $L/L_{\mathrm{q}}^{\star}$ is normalized by saved query label. PRM is built once per saved scene and reported at one cumulative roadmap checkpoint; RRTConnect and BIT* solve each query independently. BIT* rows are post-hoc quality-normalized checkpoint envelopes, not deployable online stopping policies. IRIS-NP/GCS is omitted because its archived saved-catalog context run did not yield a comparable strict-success row: all hard scenarios were 0/100, and the medium rows were 0/100 (IIWA), 30/100 (UR5), and 10/100 (Panda). The manifest mixes no-region and disconnected-source failures without separating region generation, infeasibility, and audit attribution, so an incomparable row is not mixed into the summary. The random-scene rows should be read as an online latency

boundary case: one-shot planners can be faster on individual queries because they avoid constructing a reusable region graph, while RBF can reduce registered online latency by reusing its forest and admitting strictly validated query-specific OBB corridor witnesses. In the registered profile, OBB bridge validation is no longer the dominant cost: the random-scene manifest records a 6.5 ms median per-scene OBB-cover validation cost with a 10.3 ms third quartile and 81.6 ms maximum. At the aggregate Table IV operating points, RBF is faster online than BIT* in four of the six robot–difficulty panels and has lower path ratio in the two IIWA panels; the UR5-Medium and Panda-Hard rows remain cases where the selected BIT* checkpoint is better in both displayed metrics.

D. Mechanism Studies

Tables V–VII isolate endpoint-source behavior, envelope narrow phases, and read-stage LECT replay. The UR5 snapshot gives hot endpoint/evidence reads of 0.577/0.132 μs; "open snapshot" is handle setup, not load-and-resident-memory accounting. Negative gaps in Table V are measured only against a finite sampling-union diagnostic. They can expose empirical under-coverage for sampled or diagnostic sources, but only IFK/HIFK rows are treated as certificate-backed; Analytical is a diagnostic/reference computation and is not used as a safety guarantee. Table VI motivates the AABB broadphase: SupportHull/GJK is compact, but wide boxes become expensive when every obstacle reaches GJK, as measured again in table II.

E. Dynamic Updates

The dynamic-update table isolates scene-cache maintenance only: no query bridge, connector, simplification, audit, or query timing is included. Median insert/remove costs are 0.044 [0.031, 0.050] s and 0.052 [0.034, 0.071] s, giving 39.0 [35.9, 55.9]× and 35.0 [24.5, 47.7]× maintenance speedups over the 15-obstacle warm rebuild. A matched update+query diagnostic returned 0/8 audited paths for both updated and fresh maintenance-only forests, so this is not replanning evidence.

VII. Conclusion

This paper presented RBF, a link-envelope-guided box-forest planner for multi-query manipulation. RBF lifts endpoint-to-link swept-capsule bounds to C-space box validation, grows accepted boxes into a scene-level corridor graph, and keeps scene labels separate from LECT's kinematics-keyed evidence cache. The certificate is conditional: paths inside conservative box unions and validated OBB corridor regions inherit it, while repaired or post-processed paths require query validation. Under this policy, the warm-cache Shelf+IIWA row gives a 0.168 [0.167, 0.170] s reusable build excluding external snapshot precompute and 0.049 [0.048, 0.049] s five-query amortized time; the other studies show selected random-scene context points, microsecond-scale warm LECT replay, and 35.0–39.0× median isolated maintenance speedups. The scope is deliberately narrow: the tables are final-validated, not conservative-only or repair-free; one-shot planners can be faster on individual queries; load/RSS and full LECT break-even remain unmeasured; and self-collision, attached objects, grippers, and task-specific forbidden regions require additional validation policy before they inherit the certificate.

References

[1] R. Deits and R. Tedrake, "Computing large convex regions of obstacle-free space through semidefinite programming," in *Algorithmic Foundations of Robotics XI*, 2015, pp. 109–124.

[2] T. Marcucci, M. Petersen, D. von Wrangel, and R. Tedrake, "Motion planning around obstacles with convex optimization," *Sci. Robot.*, vol. 8, no. 84, 2023.

[3] T. Marcucci, J. Umenberger, P. Parrilo, and R. Tedrake, "Shortest paths in graphs of convex sets," *SIAM J. Optim.*, vol. 34, no. 1, pp. 507–532, 2024.

[4] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2006.

[5] ——, "Rapidly-exploring random trees: A new tool for path planning," Iowa State University, Tech. Rep. TR 98-11, 1998.

[6] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE Trans. Robot. Autom.*, vol. 12, no. 4, pp. 566–580, 1996.

[7] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *Int. J. Robot. Res.*, vol. 30, no. 7, pp. 846–894, 2011.

[8] A. Amice, H. Dai, P. Werner, A. Zhang, and R. Tedrake, "Finding and optimizing certified, collision-free regions in configuration space for robot manipulators," in *Algorithmic Foundations of Robotics XV*. Springer, 2022, pp. 328–348.

[9] M. Petersen and R. Tedrake, "Growing convex collision-free regions in configuration space using nonlinear programming," 2023. [Online]. Available: https://arxiv.org/abs/2303.14737

[10] P. Werner, A. Amice, T. Marcucci, D. Rus, and R. Tedrake, "Approximating robot configuration spaces with few convex sets using clique covers of visibility graphs," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 10 359–10 365.

[11] D. Blackmore and M. C. Leu, "Analysis of swept volume via Lie groups and differential equations," *Int. J. Robot. Res.*, vol. 11, no. 6, pp. 516–537, 1992.

[12] K. Abdel-Malek, J. Yang, D. Blackmore, and K. Joy, "Swept volumes: foundation, perspectives, and applications," *Int. J. Shape Model.*, vol. 12, no. 1, pp. 87–127, 2006.

[13] S. Gottschalk, M. C. Lin, and D. Manocha, "OBBTree: A hierarchical structure for rapid interference detection," in *ACM SIGGRAPH*, 1996.

[14] E. Larsen, S. Gottschalk, M. C. Lin, and D. Manocha, "Fast proximity queries with swept sphere volumes," University of North Carolina at Chapel Hill, Tech. Rep. TR99-018, 1999.

[15] E. G. Gilbert, D. W. Johnson, and S. S. Keerthi, "A fast procedure for computing the distance between complex objects in three-dimensional space," *IEEE J. Robot. Autom.*, vol. 4, no. 2, pp. 193–203, 1988.

[16] G. v. d. Bergen, "A fast and robust GJK implementation for collision detection of convex objects," *J. Graph. Tools*, vol. 4, no. 2, pp. 7–25, 1999.

[17] J. Pan, S. Chitta, and D. Manocha, "FCL: A general purpose library for collision and proximity queries," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2012, pp. 3859–3866.

[18] R. E. Moore, R. B. Kearfott, and M. J. Cloud, *Introduction to Interval Analysis*. SIAM, 2009.

[19] L. Jaulin, M. Kieffer, O. Didrit, and E. Walter, *Applied Interval Analysis*. Springer, 2001.

[20] J. M. Snyder, "Interval analysis for computer graphics," in *Proc. ACM SIGGRAPH*, 1992, pp. 121–130.

[21] J.-P. Merlet, "Solving the forward kinematics of a Gough-type parallel manipulator with interval analysis," *Int. J. Robot. Res.*, vol. 23, no. 3, pp. 221–235, 2004.

[22] ——, "Interval analysis for certified numerical solution of problems in robotics," *Int. J. Appl. Math. Comput. Sci.*, vol. 19, no. 3, pp. 399–412, 2009.

[23] S. Redon, Y. J. Kim, M. C. Lin, and D. Manocha, "Interactive and continuous collision detection for avatars in virtual environments," in *IEEE Virtual Reality Conference*, 2004, pp. 117–124.

[24] S. Redon, M. C. Lin, D. Manocha, and Y. J. Kim, "Fast continuous collision detection for articulated models," *J. Comput. Inf. Sci. Eng.*, vol. 5, no. 2, pp. 126–137, 2005.

[25] X. Zhang, S. Redon, M. Lee, and Y. J. Kim, "Continuous collision detection for articulated models using Taylor models and temporal culling," *ACM Trans. Graph.*, vol. 26, no. 3, p. 15, 2007.

[26] F. Schwarzer, M. Saha, and J.-C. Latombe, "Adaptive dynamic collision checking for single and multiple articulated robots in complex environments," *IEEE Trans. Robot.*, vol. 21, no. 3, pp. 338–353, 2005.

[27] J. A. Corrales, F. A. Candelas, and F. Torres, "Safe human–robot interaction based on dynamic sphere-swept line bounding volumes," *Robot. Comput.-Integr. Manuf.*, vol. 27, no. 1, pp. 177–185, 2011.

[28] R. Bohlin and L. E. Kavraki, "Path planning using lazy PRM," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2000, pp. 521–528.

[29] C. M. Dellin and S. S. Srinivasa, "A unifying formalism for shortest path problems with expensive edge evaluations via lazy best-first search over paths with edge selectors," in *International Conference on Automated Planning and Scheduling (ICAPS)*, 2016.

[30] E. Schmerling, L. Janson, and M. Pavone, "Optimal sampling-based motion planning under differential constraints: The driftless case," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2015.

[31] D. Coleman, I. Sucan, S. Chitta, and N. Correll, "Reducing the barrier to entry of complex robotic software: A MoveIt! case study," *J. Softw. Eng. Robot.*, vol. 5, no. 1, pp. 3–16, 2014.

[32] T. Lozano-Pérez, "Spatial planning: A configuration space approach," *IEEE Trans. Comput.*, vol. C-32, no. 2, pp. 108–120, 1983.

[33] R. A. Brooks and T. Lozano-Pérez, "A subdivision algorithm in configuration space for findpath with rotation," *IEEE Trans. Syst. Man Cybern.*, vol. SMC-15, no. 2, pp. 224–233, 1985.

[34] J. J. Kuffner and S. M. LaValle, "RRT-connect: An efficient approach to single-query path planning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2000, pp. 995–1001.

[35] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Batch informed trees (BIT*): Sampling-based optimal planning via the heuristically guided search of implicit random geometric graphs," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2015, pp. 3067–3074.

[36] J. D. Gammell, T. D. Barfoot, and S. S. Srinivasa, "Informed sampling for asymptotically optimal path planning," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 966–984, 2018.

[37] L. Janson, E. Schmerling, A. Clark, and M. Pavone, "Fast marching tree: A fast marching sampling-based method for optimal motion planning in many dimensions," *Int. J. Robot. Res.*, vol. 34, no. 7, pp. 883–921, 2015.

[38] I. A. Şucan, M. Moll, and L. E. Kavraki, "The open motion planning library," *IEEE Robot. Autom. Mag.*, vol. 19, no. 4, pp. 72–82, 2012.

[39] M. Moll, I. A. Şucan, and L. E. Kavraki, "Benchmarking motion planning algorithms: An extensible infrastructure for analysis and visualization," *IEEE Robot. Autom. Mag.*, vol. 22, no. 3, pp. 96–102, 2015.

[40] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, "CHOMP: Gradient optimization techniques for efficient motion planning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2009, pp. 489–494.

[41] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, "STOMP: Stochastic trajectory optimization for motion planning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2011, pp. 4569–4574.

[42] J. Schulman, Y. Duan, J. Ho, A. Lee, I. Awwal, H. Bradlow, J. Pan, S. Patil, K. Goldberg, and P. Abbeel, "Motion planning with sequential convex optimization and convex collision checking," *Int. J. Robot. Res.*, vol. 33, no. 9, pp. 1251–1270, 2014.

[43] M. Mukadam, J. Dong, X. Yan, F. Dellaert, and B. Boots, "Continuous-time gaussian process motion planning via probabilistic inference," *Int. J. Robot. Res.*, vol. 37, no. 11, pp. 1319–1340, 2018.